



UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computación

Ingeniería en Software

PRÁCTICA EXPERIMENTAL V

Desarrollo de aplicaciones web mediante el modelo de desarrollo MVC (Modelo - Vista - Controlador)

Materia: Aplicaciones Web

Docente: Gleiston Guerrero Ulloa

Período Académico: SPA 2025-2026

Integrantes:

- Bueno Troya Erwin
- Reinoso Vélez Eduardo
- Silva Triviño John
- Zambrano Triviño Leonel

Quevedo - Ecuador
Febrero 2026

Investigación bibliográfica: Desarrollo de aplicaciones web basadas en MVC

La presente práctica experimental tiene como finalidad aplicar el modelo de desarrollo Modelo–Vista–Controlador (MVC) en la construcción de una aplicación web funcional, utilizando tecnologías del ecosistema Java como Spring Boot, Hibernate y Jakarta Server Faces (JSF). La presente investigación se enmarca en la asignatura Aplicaciones Web y tiene como propósito analizar los fundamentos teóricos y tecnológicos necesarios para el desarrollo de una aplicación web basada en el modelo MVC, empleando herramientas del ecosistema Java como Spring Boot, Hibernate y Jakarta Server Faces (JSF). Este análisis constituye la base conceptual sobre la cual se desarrollará posteriormente la implementación práctica de la aplicación.

El objetivo de esta práctica es desarrollar una aplicación web funcional aplicando el patrón arquitectónico MVC mediante el uso de Spring Boot, Hibernate y JSF, garantizando la correcta separación entre las capas de modelo, vista y controlador, así como la reutilización del código en distintos módulos del sistema.

1. Modelo arquitectónico Modelo–Vista–Controlador

El patrón Modelo–Vista–Controlador (MVC) es un modelo de diseño arquitectónico que propone la separación de una aplicación en tres componentes principales, cada uno con responsabilidades claramente definidas. Esta separación busca mejorar la organización del código, facilitar el mantenimiento del sistema y permitir el trabajo colaborativo entre diferentes equipos de desarrollo [1], [2].

El *Modelo* representa la lógica de negocio y la gestión de los datos, incluyendo las reglas que gobiernan el comportamiento del sistema. Este componente encapsula el estado de la aplicación y proporciona métodos para acceder y modificar dicha información, independientemente de la forma en que será presentada al usuario [3], [4]. La *Vista* se encarga de la presentación de la información al usuario final, mostrando los datos de forma estructurada y permitiendo la interacción con el sistema. Es responsable únicamente de la representación visual, sin contener lógica de negocio [2], [5]. El *Controlador*, por su parte, actúa como intermediario entre el modelo y la vista, gestionando las solicitudes del usuario y coordinando el flujo de la aplicación. Recibe las entradas del usuario, las procesa invocando los métodos apropiados del modelo y selecciona la vista adecuada para presentar la respuesta [6], [7].

La adopción del modelo MVC permite reducir el acoplamiento entre componentes, favorece la reutilización del código y mejora la escalabilidad de las aplicaciones web, características fundamentales en entornos de desarrollo modernos. Además, facilita la implementación de pruebas automatizadas, ya que cada capa puede ser evaluada de forma independiente [1], [6].

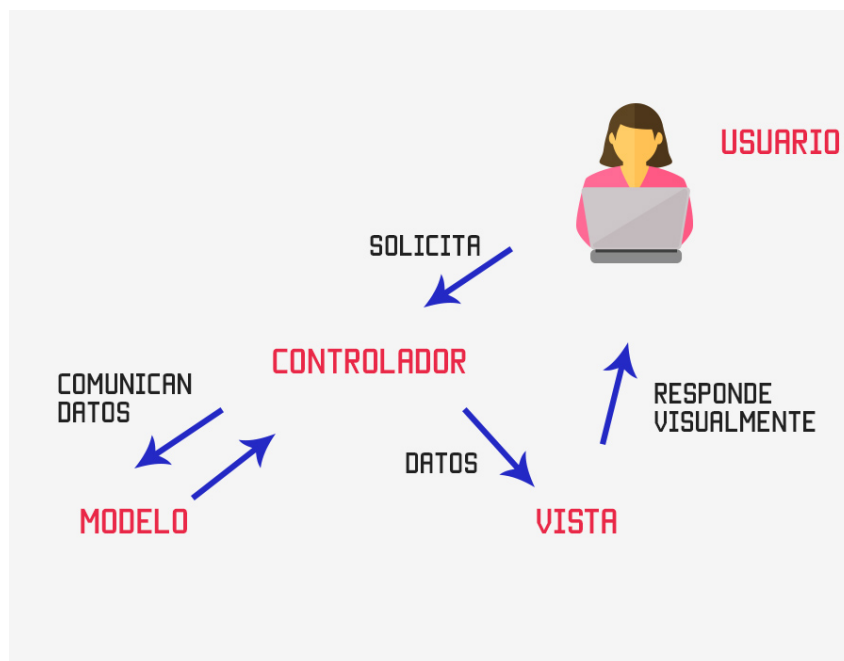


Figura 1: Diagrama conceptual del patrón Modelo-Vista-Controlador mostrando la interacción entre sus componentes principales.

1.1. Ventajas del uso del patrón MVC

La implementación del patrón MVC en aplicaciones web proporciona múltiples beneficios que han sido ampliamente documentados en la literatura científica. Entre las ventajas más significativas se encuentra la *separación de preocupaciones* (separation of concerns), que permite que diferentes equipos trabajen simultáneamente en distintas capas sin interferencias [2], [4]. Esta característica reduce significativamente los conflictos durante el desarrollo colaborativo y acelera los ciclos de entrega de software.

Otra ventaja fundamental es la *reutilización de componentes*, especialmente del modelo de datos, que puede ser compartido entre múltiples vistas sin necesidad de duplicar código [3], [7]. Esto no solo reduce el tiempo de desarrollo, sino que también minimiza la posibilidad de inconsistencias en la lógica de negocio. Adicionalmente, el patrón MVC facilita la *mantenibilidad* del software, ya que los cambios en la interfaz de usuario no requieren modificaciones en la lógica de negocio, y viceversa [1], [6].

La *testabilidad* también se ve significativamente mejorada, dado que cada componente puede ser probado de forma aislada mediante pruebas unitarias específicas [5], [6]. Por último, el patrón MVC promueve la *escalabilidad*, permitiendo que las aplicaciones crezcan en funcionalidad y usuarios sin comprometer la estructura del sistema [8], [9].

1.2. Arquitectura de aplicaciones web

La arquitectura de aplicaciones web modernas se fundamenta en principios de diseño que buscan maximizar la eficiencia, escalabilidad y mantenibilidad del software. Estos principios incluyen la organización en capas, la distribución de responsabilidades y la aplicación de patrones de diseño probados que faciliten el desarrollo iterativo y la evolución del sistema a lo largo del tiempo [2], [7].

1.2.1. Arquitectura en capas

La arquitectura en capas es un patrón estructural que organiza el sistema en niveles jerárquicos, donde cada capa tiene responsabilidades específicas y se comunica únicamente con las capas adyacentes. En el contexto de aplicaciones web empresariales, típicamente se identifican las siguientes capas: *presentación*, *lógica de negocio*, *acceso a datos* y *persistencia* [5], [6].

La capa de presentación gestiona la interacción con el usuario, recibiendo entradas y mostrando salidas de forma comprensible. La capa de lógica de negocio encapsula las reglas y procesos que definen el comportamiento del dominio de la aplicación [3], [4]. La capa de acceso a datos abstrae las operaciones de lectura y escritura sobre el sistema de almacenamiento, mientras que la capa de persistencia se encarga de la gestión física de los datos en bases de datos relacionales o no relacionales [8], [10].

Este enfoque estratificado facilita la modularidad, permite reemplazar implementaciones específicas sin afectar otras capas y mejora la comprensión del sistema al establecer límites claros entre diferentes áreas de responsabilidad [2], [7].

1.2.2. Separación de responsabilidades

El principio de separación de responsabilidades (Separation of Concerns, SoC) es fundamental en el diseño de software y establece que cada módulo o componente debe tener una única razón para cambiar, enfocándose en un aspecto específico de la funcionalidad del sistema [3], [4]. Este principio se relaciona directamente con el Single Responsibility Principle (SRP) de los principios SOLID, que son guías fundamentales para el desarrollo orientado a objetos.

En el contexto del patrón MVC, la separación de responsabilidades se manifiesta en la clara delimitación entre la gestión de datos (modelo), la presentación (vista) y el control del flujo (controlador) [1], [2]. Esta separación permite que los desarrolladores especializados en interfaz de usuario trabajen en las vistas sin necesidad de comprender en profundidad la lógica de negocio, mientras que los expertos en el dominio pueden enfocarse en el modelo sin preocuparse por aspectos de presentación [6].

1.2.3. Reutilización y mantenibilidad del código

La reutilización de código es uno de los objetivos principales de la ingeniería de software moderna, ya que reduce costos de desarrollo, minimiza errores y acelera la entrega de nuevas funcionalidades [3], [4]. El patrón MVC facilita la reutilización al permitir que un mismo modelo de datos sea utilizado por múltiples vistas y controladores, evitando la duplicación de lógica de negocio.

La mantenibilidad, por su parte, se refiere a la facilidad con la que un sistema puede ser modificado para corregir defectos, mejorar el rendimiento o adaptarse a nuevos requisitos [1], [6]. Un código bien estructurado siguiendo el patrón MVC presenta alta cohesión dentro de cada componente y bajo acoplamiento entre componentes, características que facilitan significativamente las tareas de mantenimiento [2], [7].

2. Jakarta Server Faces (JSF)

Jakarta Server Faces es un framework de desarrollo de interfaces de usuario para aplicaciones web Java que forma parte de la especificación Jakarta EE (anteriormente Java EE). JSF facilita la construcción de aplicaciones web basadas en componentes reutilizables y proporciona un modelo de programación orientado a eventos que simplifica el desarrollo de interfaces complejas [11], [12].

2.1. Definición y características de JSF

JSF es un framework MVC que permite a los desarrolladores construir aplicaciones web mediante componentes de interfaz de usuario reutilizables, siguiendo un enfoque declarativo basado en tecnologías como Facelets (el lenguaje de plantillas por defecto) [11], [12]. A diferencia de otros frameworks web que requieren la manipulación directa del HTML y el JavaScript, JSF abstrae gran parte de esta complejidad mediante componentes predefinidos y personalizables.

Entre las características principales de JSF se encuentran: el ciclo de vida bien definido de las peticiones, la gestión automática del estado de los componentes, la validación integrada de entradas de usuario, la conversión de datos entre tipos Java y representaciones textuales, y el soporte nativo para internacionalización [12], [13]. Estas características hacen de JSF una opción robusta para el desarrollo de aplicaciones empresariales que requieren interfaces ricas y mantenibles.

2.2. Arquitectura de JSF

La arquitectura de JSF se basa en un ciclo de vida de procesamiento de peticiones que consta de seis fases principales: Restore View, Apply Request Values, Process Validations, Update Model Values, Invoke Application y Render Response [11], [12]. Este ciclo garantiza que las peticiones del usuario sean procesadas de manera ordenada y predecible, facilitando la implementación de lógica de validación y conversión de datos.

Durante la fase de Restore View, el framework reconstruye el árbol de componentes de la página. En Apply Request Values, los valores enviados por el usuario son aplicados a los componentes correspondientes. La fase Process Validations ejecuta las validaciones definidas para cada componente, mientras que Update Model Values sincroniza los valores validados con el modelo de datos [12], [13]. Finalmente, Invoke Application ejecuta la lógica de negocio asociada a eventos de la aplicación, y Render Response genera la respuesta HTML que será enviada al navegador.

2.3. Componentes y vistas en JSF

Los componentes en JSF son elementos reutilizables de interfaz de usuario que encapsulan tanto la presentación como el comportamiento [11], [12]. Ejemplos de componentes básicos incluyen campos de texto, botones, tablas de datos y paneles de agrupación. Cada componente se renderiza como HTML en el navegador, pero es manipulado como un objeto Java en el servidor.

Las vistas en JSF se definen típicamente mediante archivos XHTML que utilizan la sintaxis de Facelets. Estos archivos combinan HTML estándar con etiquetas JSF que representan componentes de interfaz [12], [13]. La separación entre la definición de la

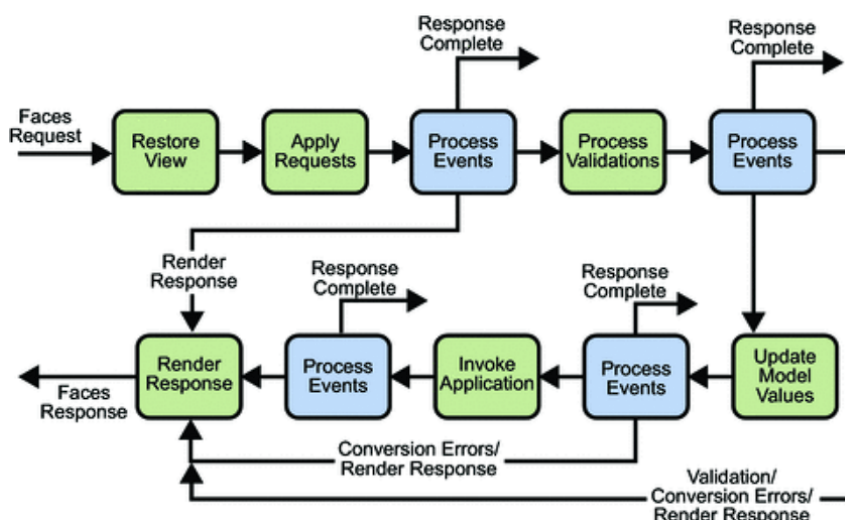


Figura 2: Ciclo de vida de procesamiento de peticiones en Jakarta Server Faces.

vista y la lógica de negocio permite que diseñadores y desarrolladores trabajen de forma independiente, mejorando la productividad del equipo.

2.4. Managed Beans y expresiones EL

Los Managed Beans son clases Java que actúan como controladores en el patrón MVC de JSF, gestionando el estado y el comportamiento de la aplicación [11], [12]. Estos beans son administrados por el contenedor JSF, que se encarga de su instanciación, configuración y ciclo de vida mediante anotaciones como `@Named` y `@RequestScoped`, `@ViewScoped`, o `@SessionScoped`.

El Expression Language (EL) es un lenguaje de expresiones que permite vincular componentes de la vista con propiedades y métodos de los Managed Beans [12], [13]. Por ejemplo, la expresión `#{usuarioBean.nombre}` en una vista XHTML puede acceder directamente a la propiedad `nombre` del bean `usuarioBean`. Esta característica facilita el desarrollo de interfaces dinámicas sin necesidad de código JavaScript complejo.

3. Hibernate como framework ORM

Hibernate es el framework de mapeo objeto-relacional (ORM) más utilizado en el ecosistema Java, permitiendo a los desarrolladores trabajar con objetos Java en lugar de consultas SQL directas para interactuar con bases de datos relacionales [10], [14]. Esta abstracción simplifica significativamente el desarrollo de la capa de persistencia y reduce los errores asociados con la manipulación manual de SQL.

3.1. Concepto de Object-Relational Mapping (ORM)

El mapeo objeto-relacional es una técnica de programación que permite convertir datos entre sistemas de tipos incompatibles: el paradigma orientado a objetos utilizado en lenguajes como Java y el modelo relacional utilizado en bases de datos [2], [8]. ORM soluciona el problema conocido como "impedance mismatch", que surge de las diferencias fundamentales entre estos dos paradigmas.

Los frameworks ORM como Hibernate generan automáticamente las consultas SQL necesarias para operaciones CRUD (Create, Read, Update, Delete), gestionan las relaciones entre entidades y optimizan el acceso a datos mediante técnicas como lazy loading y caching [10], [14]. Esto permite a los desarrolladores enfocarse en la lógica de negocio en lugar de en los detalles de implementación del acceso a datos.

3.2. Mapeo de entidades y relaciones

En Hibernate, las clases Java se mapean a tablas de base de datos mediante anotaciones JPA (Jakarta Persistence API) o archivos XML de configuración [10], [12]. Las anotaciones más comunes incluyen `@Entity` para marcar una clase como entidad persistente, `@Table` para especificar el nombre de la tabla correspondiente, `@Id` para identificar la clave primaria y `@Column` para configurar columnas específicas.

Las relaciones entre entidades se mapean mediante anotaciones como `@OneToOne`, `@OneToMany`, `@ManyToOne` y `@ManyToMany`, que especifican la cardinalidad de las asociaciones [10], [14]. Hibernate gestiona automáticamente las claves foráneas y las tablas de unión necesarias para implementar estas relaciones en la base de datos, además de proporcionar estrategias de carga (eager o lazy) para optimizar el rendimiento según las necesidades de la aplicación.

3.3. Ventajas del uso de Hibernate

El uso de Hibernate proporciona múltiples ventajas en el desarrollo de aplicaciones empresariales. En primer lugar, reduce significativamente la cantidad de código repetitivo necesario para operaciones de persistencia, aumentando la productividad del equipo de desarrollo [2], [10]. La abstracción del SQL permite que el código sea más portable entre diferentes sistemas de bases de datos, requiriendo mínimas modificaciones para cambiar de motor.

Hibernate también proporciona mecanismos avanzados de optimización como el caché de segundo nivel, que reduce las consultas a la base de datos almacenando en memoria objetos frecuentemente accedidos [8], [14]. Además, soporta consultas complejas mediante HQL (Hibernate Query Language) y Criteria API, que ofrecen mayor expresividad y seguridad de tipos que SQL nativo. La gestión automática de transacciones y el control de concurrencia mediante versionado optimista o pesimista son características adicionales que simplifican el desarrollo de aplicaciones robustas [10], [12].

4. Spring Boot

Spring Boot es un framework de desarrollo rápido basado en Spring Framework que simplifica la configuración y el despliegue de aplicaciones Java empresariales [15], [16]. Su filosofía de “convención sobre configuración” permite a los desarrolladores crear aplicaciones funcionales con mínima configuración manual, acelerando significativamente los ciclos de desarrollo.

4.1. Definición y características principales

Spring Boot proporciona una plataforma completa para el desarrollo de aplicaciones stand-alone y de grado empresarial, eliminando gran parte de la configuración XML tradicional de Spring mediante el uso extensivo de anotaciones y configuración automática [15], [17]. Entre sus características principales se encuentran: servidores embebidos (Tomcat, Jetty, Undertow) que permiten ejecutar aplicaciones como archivos JAR autónomos, configuración automática basada en las dependencias presentes en el classpath, y Spring Boot Starter POMs que agrupan dependencias comúnmente utilizadas juntas.

El framework también incluye Spring Boot Actuator, que proporciona endpoints para monitoreo y gestión de aplicaciones en producción, facilitando la observabilidad del sistema [16], [18]. Adicionalmente, Spring Boot DevTools mejora la experiencia de desarrollo mediante funcionalidades como recarga automática de código y configuraciones específicas para entornos de desarrollo.

4.2. Configuración basada en anotaciones

Spring Boot promueve el uso de anotaciones Java para configurar componentes y comportamientos de la aplicación, reemplazando los archivos XML tradicionales [15], [17]. Anotaciones como `@SpringBootApplication` marcan la clase principal de la aplicación y habilitan la configuración automática, el escaneo de componentes y la capacidad de definir beans adicionales.

Otras anotaciones fundamentales incluyen `@RestController` para definir controladores REST, `@Service` para marcar clases de lógica de negocio, `@Repository` para componentes de acceso a datos y `@Autowired` para inyección de dependencias [16], [18]. El contenedor de Spring gestiona el ciclo de vida de estos componentes y resuelve automáticamente las dependencias entre ellos, implementando el patrón de inversión de control (IoC) que caracteriza al framework.

4.3. Integración con Spring MVC y JPA

Spring Boot se integra perfectamente con Spring MVC para el desarrollo de aplicaciones web y APIs REST [15], [17]. Spring MVC implementa el patrón MVC mediante controladores anotados con `@Controller` o `@RestController`, que procesan peticiones HTTP y retornan respuestas en diversos formatos (HTML, JSON, XML). La configuración automática de Spring Boot elimina la necesidad de definir manualmente componentes como el `DispatcherServlet`, los view resolvers y los message converters.

La integración con JPA (Jakarta Persistence API) se realiza mediante Spring Data JPA, que proporciona una capa de abstracción sobre Hibernate y otros proveedores JPA [16], [19]. Spring Data JPA permite definir repositorios mediante interfaces que extienden `JpaRepository`, generando automáticamente las implementaciones de métodos CRUD sin necesidad de escribir código SQL. Además, soporta consultas derivadas del nombre del método y consultas personalizadas mediante la anotación `@Query`, combinando flexibilidad con productividad.

5. Integración de MVC con JSF, Hibernate y Spring Boot

La integración de las tecnologías JSF, Hibernate y Spring Boot permite construir aplicaciones web completas que aprovechan las fortalezas de cada framework en su área específica. Esta combinación crea un ecosistema robusto que cubre desde la persistencia de datos hasta la presentación de interfaces de usuario complejas [11], [15].

5.1. Relación entre las tecnologías

En una aplicación integrada, Spring Boot actúa como el contenedor principal que gestiona la configuración, la inyección de dependencias y el ciclo de vida de los componentes [15], [17]. Hibernate, configurado a través de Spring Data JPA, maneja la capa de persistencia y el mapeo objeto-relacional. JSF proporciona la capa de presentación, renderizando vistas dinámicas basadas en componentes y gestionando la interacción con el usuario.

La comunicación entre estas tecnologías sigue el patrón MVC: los Managed Beans de JSF actúan como controladores que invocan servicios de Spring (capa de lógica de negocio), los cuales a su vez utilizan repositorios Spring Data JPA para acceder a las entidades Hibernate [11], [16]. Esta separación de responsabilidades garantiza que cada tecnología se enfoque en su dominio específico, maximizando la cohesión y minimizando el acoplamiento.

5.2. Flujo general de una aplicación MVC

El flujo típico de una petición en una aplicación MVC integrada comienza cuando el usuario interactúa con la vista JSF, por ejemplo, enviando un formulario [11], [12]. Esta acción dispara un evento que es capturado por un Managed Bean (controlador), el cual valida los datos de entrada y delega la ejecución de la lógica de negocio a un servicio de Spring.

El servicio procesa la solicitud aplicando las reglas de negocio necesarias y, si requiere persistir o recuperar datos, invoca métodos de un repositorio Spring Data JPA [15], [19]. El repositorio interactúa con Hibernate para ejecutar las operaciones en la base de datos, ya sea mediante consultas generadas automáticamente o HQL personalizado. Los resultados son retornados al servicio, que los procesa y devuelve al controlador. Finalmente, el Managed Bean actualiza el modelo y selecciona la vista apropiada para renderizar la respuesta al usuario.

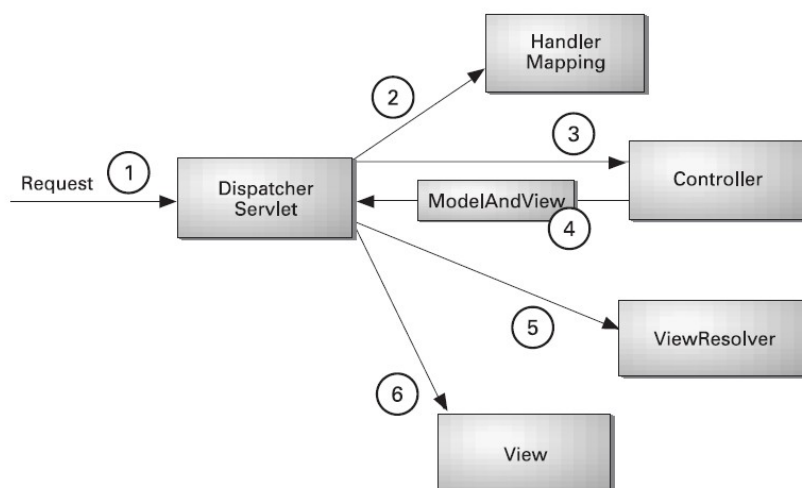


Figura 3: Flujo de procesamiento de una petición en una aplicación MVC utilizando JSF, Spring Boot e Hibernate.

5.3. Estructura lógica de controladores, servicios y repositorios

La organización del código en capas bien definidas es fundamental para mantener la claridad y escalabilidad del proyecto [2], [7]. Los *controladores* (Managed Beans en JSF o `@Controller` en Spring MVC) se encuentran en la capa de presentación y son responsables de recibir las peticiones del usuario, validar entradas básicas y coordinar la ejecución del flujo de la aplicación.

Los *servicios*, marcados con `@Service`, constituyen la capa de lógica de negocio y contienen las reglas y procesos que definen el comportamiento del dominio [3], [4]. Esta capa es transaccional y coordina operaciones que pueden involucrar múltiples repositorios. Debe ser independiente de la tecnología de presentación utilizada, permitiendo su reutilización en diferentes contextos (web, API REST, servicios batch).

Los *repositorios*, marcados con `@Repository` o extendiendo interfaces de Spring Data JPA, constituyen la capa de acceso a datos y encapsulan todas las operaciones de persistencia [16], [20]. Esta capa abstrae los detalles de Hibernate y SQL, proporcionando una interfaz limpia basada en el lenguaje del dominio. La separación clara entre estas capas facilita las pruebas unitarias, el mantenimiento y la evolución del sistema a lo largo del tiempo.

6. Conclusiones

El patrón arquitectónico Modelo–Vista–Controlador se ha consolidado como un estándar fundamental en el desarrollo de aplicaciones web modernas, proporcionando una estructura clara que separa las responsabilidades de presentación, lógica de negocio y gestión de datos. Esta separación no solo mejora la organización del código, sino que también facilita el mantenimiento, la escalabilidad y la colaboración entre equipos de desarrollo especializados.

La integración de tecnologías como Jakarta Server Faces, Hibernate y Spring Boot permite construir aplicaciones web robustas y eficientes que aprovechan las fortalezas específicas de cada framework. JSF proporciona una capa de presentación rica en componentes reutilizables con un ciclo de vida bien definido. Hibernate abstrae la complejidad del acceso a datos relacionales mediante mapeo objeto-relacional, mientras que Spring Boot simplifica la configuración y gestión del ecosistema completo mediante inyección de dependencias y configuración automática.

La arquitectura en capas resultante de esta integración promueve principios de ingeniería de software como la alta cohesión, el bajo acoplamiento y la separación de responsabilidades. Estos principios son esenciales para desarrollar sistemas mantenibles que puedan evolucionar ante cambios en los requisitos de negocio o en las tecnologías subyacentes. La comprensión profunda de estos fundamentos teóricos constituye la base necesaria para la implementación práctica exitosa de aplicaciones web empresariales.

El presente marco teórico establece las bases conceptuales sobre las cuales se desarrollará la práctica experimental, garantizando que la implementación técnica esté fundamentada en principios sólidos de diseño arquitectónico y mejores prácticas de la ingeniería de software moderna.

Desarrollo de una aplicación web basada en el modelo MVC utilizando JSF, Hibernate y SpringBoot

La presente práctica experimental tiene como finalidad aplicar el modelo arquitectónico Modelo–Vista–Controlador (MVC) en el desarrollo de una aplicación web funcional, utilizando tecnologías del ecosistema Java. Este documento corresponde a la fase práctica de la actividad y presenta la implementación del sistema, evidenciando la correcta separación entre las capas de presentación, lógica de negocio y acceso a datos.

La aplicación fue desarrollada empleando Spring Boot, Hibernate y Jakarta Server Faces (JSF), integrando dichas tecnologías para demostrar el funcionamiento del patrón MVC, así como la organización del código y la interacción entre sus componentes.

7. Objetivo de la práctica

Desarrollar una aplicación web funcional aplicando el modelo Modelo–Vista–Controlador (MVC), mediante el uso de Spring Boot, Hibernate y Jakarta Server Faces (JSF), garantizando la separación de responsabilidades entre las capas del sistema y la correcta implementación de operaciones CRUD desde la interfaz de usuario.

8. Planeación y diseño

En esta sección se describe la etapa de planeación y diseño del sistema desarrollado, la cual permite definir el alcance de la aplicación, su estructura general y los elementos principales que intervienen en su funcionamiento. Esta fase es fundamental para establecer una base clara antes de la configuración e implementación del proyecto, garantizando una correcta aplicación del modelo arquitectónico Modelo–Vista–Controlador (MVC).

8.1. Caso de estudio

El caso de estudio corresponde al desarrollo de una aplicación web orientada a la gestión básica de usuarios, productos y roles, permitiendo realizar operaciones de creación, consulta, actualización y eliminación de registros desde una interfaz web. El sistema tiene como objetivo demostrar la aplicación práctica del modelo arquitectónico Modelo–Vista–Controlador (MVC) en un entorno real de desarrollo.

La aplicación permite al usuario interactuar con formularios web para registrar información, visualizar listados de datos y ejecutar acciones sobre los registros almacenados en la base de datos. Cada acción realizada desde la vista es procesada por los controladores correspondientes, los cuales delegan la lógica de negocio a la capa de servicios y el acceso a datos a los repositorios, garantizando una adecuada separación de responsabilidades.

Este caso de estudio sirve como base para evidenciar la integración entre la capa de presentación (JSF), la lógica de negocio (Spring Boot) y la persistencia de datos (Hibernate), cumpliendo con los lineamientos establecidos en la guía de la práctica experimental.

8.2. Modelo de datos

El modelo de datos define las entidades que componen el sistema, sus atributos principales y las relaciones necesarias para asegurar consistencia e integridad en la información. Esta definición se utiliza como base para la persistencia mediante JPA/Hibernate y para soportar las operaciones CRUD implementadas desde la capa de presentación.

Para esta práctica, el sistema se estructuró en tres entidades principales: *User*, *Product* y *Role*. Cada entidad cuenta con un identificador único y atributos esenciales para su gestión. Adicionalmente, se considera la asignación de roles a usuarios para representar permisos o perfiles dentro del sistema.

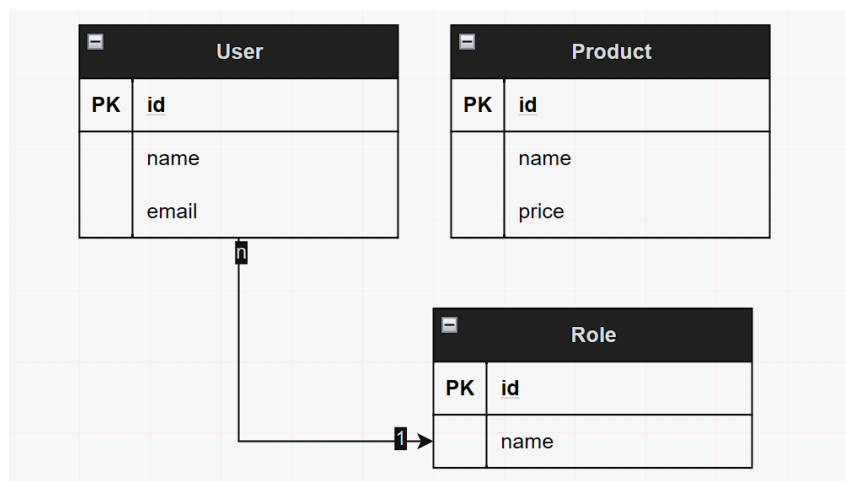


Figura 4: Modelo de datos: entidades y relaciones del sistema

Entidades definidas:

- **User:** representa a los usuarios gestionados por el sistema.
 - Atributos: **id**, **name**, **email**.
- **Product:** representa los productos administrados en la aplicación.
 - Atributos: **id**, **name**, **price**.
- **Role:** representa los roles disponibles para asignación.
 - Atributos: **id**, **name**.

Relaciones: el modelo contempla la relación entre *User* y *Role*. De acuerdo con el diseño, un usuario puede estar asociado a un rol para diferenciar responsabilidades dentro del sistema. Esta relación se refleja en el diagrama mediante una asociación entre ambas entidades y será utilizada por la capa de persistencia para mantener integridad referencial en la base de datos.

8.3. Esquema del flujo MVC

En esta subsección se presenta el flujo general de la aplicación bajo el modelo Modelo–Vista–Controlador (MVC), destacando la separación de responsabilidades entre la interfaz de usuario, el control del flujo de la aplicación, la lógica de negocio y el acceso a datos. Este

esquema permite comprender cómo se procesan las acciones del usuario y cómo se realiza la comunicación con la base de datos sin que la capa de presentación acceda directamente a la persistencia.

De forma general, las vistas desarrolladas con JSF (archivos XHTML) capturan las interacciones del usuario y envían las solicitudes al Bean correspondiente, el cual actúa como controlador. Posteriormente, el Bean delega la operación requerida a la capa de servicios, donde se centraliza la lógica de negocio. Finalmente, el servicio utiliza la capa de repositorios para ejecutar las operaciones de persistencia mediante JPA/Hibernate en la base de datos PostgreSQL. La respuesta retorna en sentido inverso, actualizando el estado mostrado en la vista.

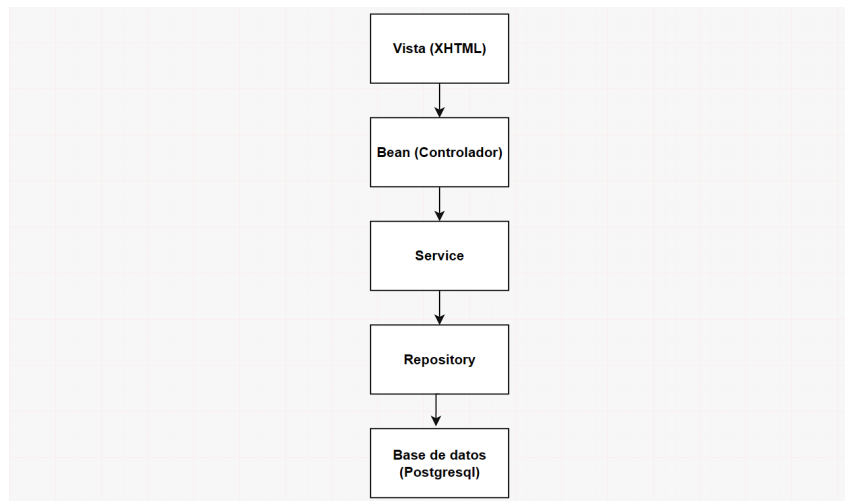


Figura 5: Flujo MVC por capas: Vista (JSF) → Bean → Service → Repository → Base de datos

9. Configuración del proyecto

En esta sección se describen los pasos de configuración realizados para preparar el entorno de desarrollo, integrar las dependencias necesarias y establecer la conexión con la base de datos. Esta configuración permite asegurar que la aplicación pueda ejecutarse correctamente bajo una arquitectura MVC, soportando la capa de presentación con JSF, la lógica con Spring Boot y la persistencia con Hibernate/JPA.

9.1. Creación del proyecto

El proyecto fue creado en IntelliJ IDEA utilizando Spring Boot como base para la estructura inicial de la aplicación. Se seleccionó Java 17 por su estabilidad y compatibilidad con las versiones utilizadas del framework. Para el proceso de construcción y gestión de dependencias se empleó Maven, lo cual facilita la integración de librerías externas y el control de versiones de forma centralizada.

Durante la creación inicial, el asistente del IDE generó el proyecto con una versión reciente de Spring Boot. Sin embargo, debido a consideraciones de compatibilidad entre Spring Boot 3.x y bibliotecas de JSF/JoinFaces (por la transición de paquetes hacia Jakarta EE), se ajustó la versión del *parent* en el archivo `pom.xml` a una versión estable compatible con la integración JSF.

```

1 <parent>
2   <groupId>org.springframework.boot</groupId>
3   <artifactId>spring-boot-starter-parent</artifactId>
4   <version>2.7.18</version>
5   <relativePath/>
6 </parent>

```

Listing 1: Configuración del parent de Spring Boot en pom.xml

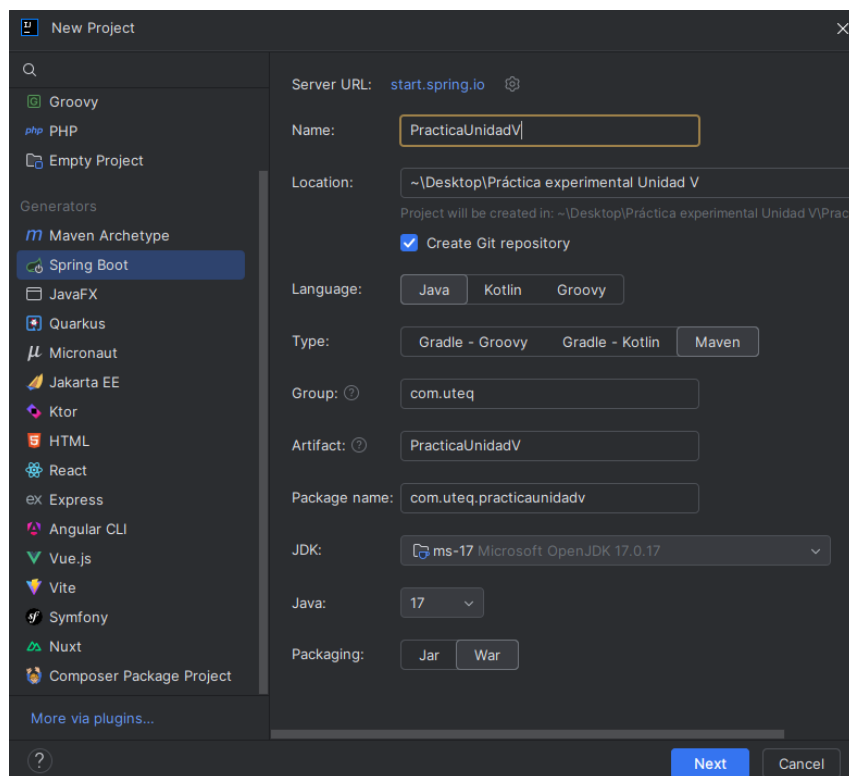


Figura 6: Creación del proyecto y configuración inicial en el IDE

9.2. Dependencias del sistema

Para el correcto funcionamiento de la aplicación web fue necesario incorporar un conjunto de dependencias que permiten la integración entre la capa de presentación, la lógica de negocio y el acceso a datos. Estas dependencias fueron gestionadas mediante Maven, facilitando su configuración y mantenimiento desde el archivo `pom.xml`.

Las librerías seleccionadas permiten implementar el patrón MVC de forma completa, integrando Spring Boot como framework principal, Jakarta Server Faces (JSF) para la interfaz de usuario y Hibernate/JPA para la persistencia de datos en una base de datos PostgreSQL.

```

1 <dependencies>
2
3   <!-- Spring Web -->
4   <dependency>
5       <groupId>org.springframework.boot</groupId>
6       <artifactId>spring-boot-starter-web</artifactId>
7   </dependency>
8
9   <!-- Spring Data JPA -->
10  <dependency>
11      <groupId>org.springframework.boot</groupId>
12      <artifactId>spring-boot-starter-data-jpa</artifactId>
13  </dependency>
14
15  <!-- PostgreSQL Driver -->
16  <dependency>
17      <groupId>org.postgresql</groupId>
18      <artifactId>postgresql</artifactId>
19      <scope>runtime</scope>
20  </dependency>
21
22  <!-- JoinFaces: integracion JSF + Spring Boot -->
23  <dependency>
24      <groupId>org.joinfaces</groupId>
25      <artifactId>jsf-spring-boot-starter</artifactId>
26      <version>4.7.12</version>
27  </dependency>
28
29  <!-- PrimeFaces -->
30  <dependency>
31      <groupId>org.primefaces</groupId>
32      <artifactId>primefaces</artifactId>
33      <version>12.0.0</version>
34  </dependency>
35
36  <!-- Inyeccion de dependencias -->
37  <dependency>
38      <groupId>javax.inject</groupId>
39      <artifactId>javax.inject</artifactId>
40      <version>1</version>
41  </dependency>
42
43 </dependencies>

```

Listing 2: Dependencias principales del proyecto en pom.xml

Cada una de estas dependencias cumple un rol específico dentro de la arquitectura del sistema. Spring Boot simplifica la configuración general del proyecto, JSF permite construir interfaces basadas en componentes, Hibernate gestiona la persistencia de datos y PostgreSQL actúa como motor de base de datos relacional, garantizando un entorno estable y coherente para la ejecución de la aplicación.

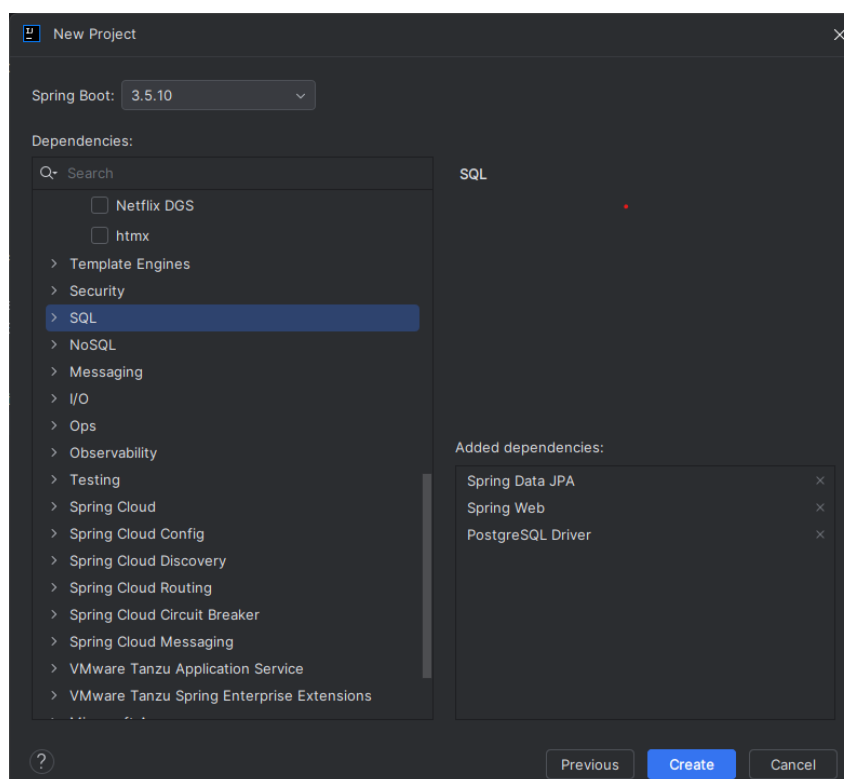


Figura 7: Configuración de dependencias del proyecto

9.3. Configuración de la base de datos

La configuración de la base de datos es un paso fundamental para permitir la persistencia de la información gestionada por el sistema. En esta práctica se utilizó PostgreSQL como sistema gestor de base de datos relacional, debido a su estabilidad, compatibilidad con Hibernate y amplio uso en aplicaciones empresariales.

La conexión entre la aplicación y la base de datos se configuró mediante el archivo `application.properties`, donde se definieron los parámetros necesarios para establecer la comunicación, así como las opciones de Hibernate para la gestión automática del esquema de la base de datos.

```

1 spring.application.name=PracticaUnidadV
2 server.port=8080
3
4 spring.jpa.database=postgresql
5 spring.jpa.show-sql=false
6 spring.jpa.hibernate.ddl-auto=update
7 spring.jpa.database-platform=org.hibernate.dialect.PostgreSQLDialect
8
9 spring.datasource.driver-class-name=org.postgresql.Driver
10 spring.datasource.url=jdbc:postgresql://localhost:5432/practicaunidadv
11 spring.datasource.username=postgres
12 spring.datasource.password=12072000
13
14 joinfaces.faces-servlet.url-mappings=*.xhtml
15 server.servlet.context-path=/

```

Listing 3: Configuración de la conexión a la base de datos en `application.properties`

La propiedad `ddl-auto=update` permite que Hibernate cree y actualice automáticamente

mente las tablas en función de las entidades definidas en el sistema, facilitando el desarrollo y evitando configuraciones manuales adicionales. De esta manera, las entidades del modelo de datos se reflejan directamente en la estructura de la base de datos.

9.4. Arquitectura del proyecto

La arquitectura del proyecto se organizó siguiendo una estructura por capas, con el objetivo de separar responsabilidades y facilitar la mantenibilidad del sistema. Esta organización permite que cada componente cumpla una función específica dentro de la aplicación, alineándose con el modelo arquitectónico Modelo–Vista–Controlador (MVC).

El proyecto se estructuró en distintos paquetes, cada uno asociado a una capa del sistema. Esta distribución favorece la reutilización del código, mejora la legibilidad del proyecto y permite realizar modificaciones en una capa sin afectar directamente a las demás.

- **entity**: contiene las clases que representan las entidades del modelo de datos y se mapean directamente a las tablas de la base de datos mediante JPA/Hibernate.
- **repository**: incluye las interfaces responsables del acceso a datos, encargadas de ejecutar las operaciones de persistencia utilizando Spring Data JPA.
- **service**: agrupa la lógica de negocio del sistema, actuando como intermediario entre los controladores y la capa de acceso a datos.
- **bean**: contiene los Beans de JSF que funcionan como controladores, gestionando las acciones del usuario desde la vista.
- **templates**: almacena las plantillas y componentes reutilizables de la interfaz gráfica desarrollados con Facelets.
- **roleConverter**: contiene el convertidor de roles utilizado por componentes JSF, permitiendo transformar el valor seleccionado en la vista (String) al objeto *Role* correspondiente.

Esta arquitectura permite que las vistas no interactúen directamente con la base de datos, garantizando que toda operación pase por las capas correspondientes. De esta manera, se asegura una correcta aplicación del patrón MVC y una estructura ordenada del sistema.

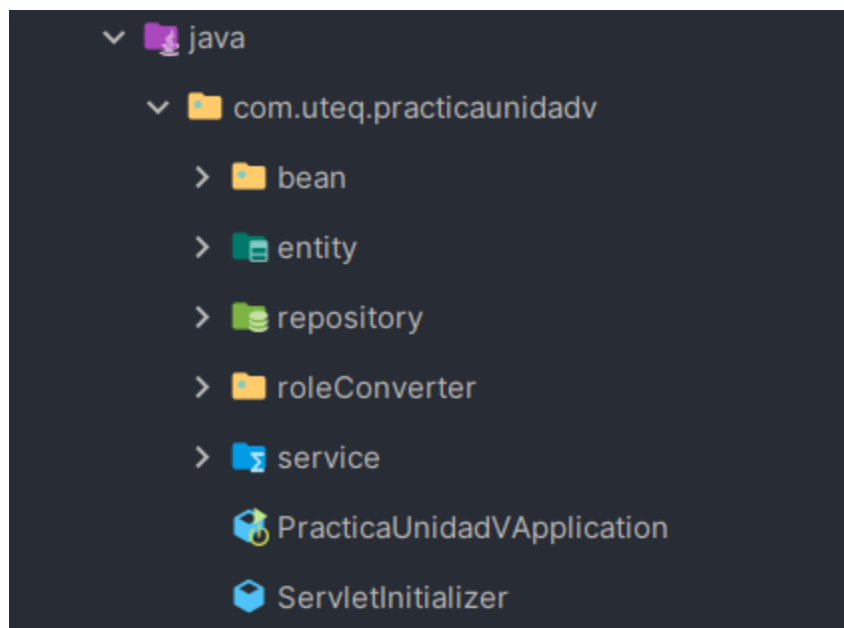


Figura 8: Estructura del proyecto organizada por capas

10. Implementación del sistema

En esta sección se describe la implementación técnica de los componentes principales del sistema, aplicando la arquitectura definida previamente. Se inicia con la capa de modelo, donde se representan las entidades del sistema y su mapeo a la base de datos mediante JPA/Hibernate.

10.1. Modelo: entidades JPA

Las entidades JPA representan el modelo de datos del sistema y permiten mapear las tablas de la base de datos a objetos Java mediante Hibernate. Cada entidad se define utilizando anotaciones de JPA, lo que facilita la persistencia de la información y la ejecución de operaciones CRUD sin necesidad de escribir consultas SQL manuales.

En esta práctica se implementaron las entidades *User*, *Product* y *Role*. La entidad *User* mantiene una relación con *Role*, lo que permite asociar a cada usuario un rol específico dentro del sistema. Esta relación refuerza el control estructural de los datos y se gestiona directamente desde el modelo.

```
1 package com.uteq.practicaunidadv.entity;
2
3 import javax.persistence.*;
4
5 @Entity
6 @Table(name = "users")
7 public class User {
8
9     @Id
10    @GeneratedValue(strategy = GenerationType.IDENTITY)
11    private Long id;
12
13    @Column(nullable = false)
14    private String name;
15
16    @Column(nullable = false, unique = true)
17    private String email;
18
19    @ManyToOne(optional = false)
20    @JoinColumn(name = "id_rol", nullable = false)
21    private Role role;
22
23    public User() {}
24
25    public Long getId() { return id; }
26    public void setId(Long id) { this.id = id; }
27
28    public String getName() { return name; }
29    public void setName(String name) { this.name = name; }
30
31    public String getEmail() { return email; }
32    public void setEmail(String email) { this.email = email; }
33
34    public Role getRole() { return role; }
35    public void setRole(Role role) { this.role = role; }
36 }
```

Listing 4: Entidad User

```

1 package com.uteq.practicaunidadv.entity;
2
3 import javax.persistence.*;
4
5 @Entity
6 public class Product {
7
8     @Id
9     @GeneratedValue(strategy = GenerationType.IDENTITY)
10    private Long id;
11
12    private String name;
13    private Double price;
14
15    public Product() {}
16
17    public Long getId() { return id; }
18    public void setId(Long id) { this.id = id; }
19
20    public String getName() { return name; }
21    public void setName(String name) { this.name = name; }
22
23    public Double getPrice() { return price; }
24    public void setPrice(Double price) { this.price = price; }
25 }

```

Listing 5: Entidad Product

```

1 package com.uteq.practicaunidadv.entity;
2
3 import javax.persistence.*;
4
5 @Entity
6 public class Role {
7
8     @Id
9     @GeneratedValue(strategy = GenerationType.IDENTITY)
10    private Long id;
11
12    private String name;
13
14    public Role() {}
15
16    public Long getId() { return id; }
17    public void setId(Long id) { this.id = id; }
18
19    public String getName() { return name; }
20    public void setName(String name) { this.name = name; }
21 }

```

Listing 6: Entidad Role

La relación entre *User* y *Role* se implementó como una asociación de muchos a uno, donde varios usuarios pueden compartir un mismo rol. Esta decisión de diseño es coherente con el alcance de la práctica y permite demostrar el manejo de relaciones entre entidades dentro del modelo de persistencia.

10.2. Repositorio genérico

Para evitar la duplicación de código y facilitar la reutilización de las operaciones básicas de persistencia, se implementó un repositorio genérico que centraliza las funcionalidades CRUD comunes a todas las entidades del sistema. Este enfoque permite que las entidades compartan una base común para el acceso a datos, reduciendo la complejidad y mejorando la mantenibilidad del proyecto.

El repositorio genérico se define mediante la interfaz `BaseRepository`, la cual extiende `JpaRepository` de Spring Data JPA. Al declararse como `@NoRepositoryBean`, se indica que esta interfaz no debe ser instanciada directamente, sino heredada por repositorios específicos de cada entidad.

```

1 package com.uteq.practicaunidadv.repository;
2
3 import org.springframework.data.jpa.repository.JpaRepository;
4 import org.springframework.data.repository.NoRepositoryBean;
5
6 @NoRepositoryBean
7 public interface BaseRepository<T, ID>
8     extends JpaRepository<T, ID> {
9 }

```

Listing 7: Repositorio genérico `BaseRepository`

A partir de este repositorio base, se crearon repositorios específicos para cada entidad del sistema, los cuales heredan automáticamente las operaciones CRUD sin necesidad de redefinir métodos adicionales. Esto permite que la capa de servicios interactúe con los datos de manera uniforme, independientemente de la entidad gestionada.

Como ejemplo, el repositorio correspondiente a la entidad *User* extiende directamente del repositorio genérico, heredando todas sus funcionalidades.

```

1 package com.uteq.practicaunidadv.repository;
2
3 import com.uteq.practicaunidadv.entity.User;
4
5 public interface UserRepository
6     extends BaseRepository<User, Long> {
7 }

```

Listing 8: Repositorio específico `UserRepository`

Este diseño refuerza la arquitectura por capas del sistema y garantiza que el acceso a datos se realice de forma consistente, alineándose con buenas prácticas de desarrollo de software.

10.3. Servicios genéricos

Con el fin de centralizar la lógica común y mantener la coherencia en la gestión de entidades, se implementó una capa de servicios genérica. Esta capa permite reutilizar operaciones frecuentes como guardar, listar, buscar por identificador y eliminar, evitando repetir la misma lógica en múltiples servicios específicos.

El servicio genérico se definió mediante la interfaz `BaseService`, donde se declararon las operaciones básicas que serán compartidas por todas las entidades del sistema.

```

1 package com.uteq.practicaunidadv.service;
2
3 import java.util.List;
4
5 public interface BaseService<T, ID> {
6
7     T save(T entity);
8
9     List<T> findAll();
10
11     void delete(T entity);
12
13     T findById(ID id);
14 }

```

Listing 9: Interfaz BaseService

Posteriormente, se implementó la clase abstracta `BaseServiceImpl`, la cual proporciona la implementación de los métodos definidos en `BaseService`. Esta clase delega las operaciones al repositorio correspondiente mediante el método abstracto `getRepository()`, el cual es implementado por cada servicio específico.

```

1 package com.uteq.practicaunidadv.service.impl;
2
3 import com.uteq.practicaunidadv.repository.BaseRepository;
4 import com.uteq.practicaunidadv.service.BaseService;
5 import java.util.List;
6
7 public abstract class BaseServiceImpl<T, ID>
8     implements BaseService<T, ID> {
9
10     protected abstract BaseRepository<T, ID> getRepository();
11
12     @Override
13     public T save(T entity) {
14         return getRepository().save(entity);
15     }
16
17     @Override
18     public List<T> findAll() {
19         return getRepository().findAll();
20     }
21
22     @Override
23     public void delete(T entity) {
24         getRepository().delete(entity);
25     }
26
27     @Override
28     public T findById(ID id) {
29         return getRepository().findById(id).orElse(null);
30     }
31 }

```

Listing 10: Implementación genérica BaseServiceImpl

A partir de esta estructura, se definieron servicios específicos por entidad que heredan la implementación genérica. Por ejemplo, el servicio de *User* extiende `BaseServiceImpl` e inyecta el repositorio correspondiente.

```

1 package com.uteq.practicaunidadv.service;
2
3 import com.uteq.practicaunidadv.entity.User;
4 import com.uteq.practicaunidadv.repository.BaseRepository;
5 import com.uteq.practicaunidadv.repository.UserRepository;
6 import com.uteq.practicaunidadv.service.impl.BaseServiceImpl;
7 import org.springframework.stereotype.Service;
8
9 @Service
10 public class UserService
11     extends BaseServiceImpl<User, Long> {
12
13     private final UserRepository userRepository;
14
15     public UserService(UserRepository userRepository) {
16         this.userRepository = userRepository;
17     }
18
19     @Override
20     protected BaseRepository<User, Long> getRepository() {
21         return userRepository;
22     }
23 }

```

Listing 11: Servicio específico UserService

El uso de servicios genéricos mejora la mantenibilidad del sistema, ya que permite implementar nuevas entidades reutilizando la misma lógica base, conservando una arquitectura consistente y alineada con buenas prácticas.

10.4. Controladores JSF (Beans)

Los controladores del sistema se implementaron mediante Beans administrados de JSF, los cuales actúan como intermediarios entre la capa de presentación y la lógica de negocio. Estos Beans reciben las acciones del usuario desde las vistas XHTML, procesan los eventos correspondientes y delegan las operaciones a la capa de servicios.

En esta práctica, los Beans se anotaron como componentes de Spring y se definieron con alcance `@ViewScoped`, lo que permite mantener el estado de la vista mientras el usuario interactúa con ella. De esta manera, se facilita la implementación de funcionalidades como edición de registros y actualización dinámica de datos.

A continuación, se presenta el Bean correspondiente a la gestión de usuarios, el cual permite realizar las operaciones CRUD desde la interfaz gráfica.

```

1 package com.uteq.practicaunidadv.bean;
2
3 import com.uteq.practicaunidadv.entity.Role;
4 import com.uteq.practicaunidadv.entity.User;
5 import com.uteq.practicaunidadv.service.RoleService;
6 import com.uteq.practicaunidadv.service.UserService;
7 import org.springframework.stereotype.Component;
8
9 import javax.faces.view.ViewScoped;
10 import javax.inject.Inject;
11 import javax.faces.application.FacesMessage;
12 import javax.faces.context.FacesContext;
13

```



```

14 import java.io.Serializable;
15 import java.util.List;
16
17 @Component
18 @ViewScoped
19 public class UserBean implements Serializable {
20
21     @Inject
22     private UserService userService;
23
24     @Inject
25     private RoleService roleService;
26
27     private User user = new User();
28     private boolean editing = false;
29
30     public void save() {
31         try {
32
33             boolean wasEditing = editing;
34
35             userService.save(user);
36
37             FacesContext.getCurrentInstance().addMessage(null,
38                 new FacesMessage(
39                     FacesMessage.SEVERITY_INFO,
40                     null,
41                     wasEditing
42                         ? "Usuario actualizado correctamente"
43                         : "Usuario guardado correctamente"
44                 ));
45
46             user = new User();
47             editing = false;
48
49         } catch (Exception e) {
50
51             FacesContext.getCurrentInstance().addMessage(null,
52                 new FacesMessage(
53                     FacesMessage.SEVERITY_ERROR,
54                     null,
55                     "Error al guardar el usuario"
56                 ));
57         }
58     }
59
60     public List<User> getUsers() {
61         return userService.findAll();
62     }
63
64     public void delete(User user) {
65         try {
66
67             userService.delete(user);
68
69             FacesContext.getCurrentInstance().addMessage(null,
70                 new FacesMessage(

```

```

71         FacesMessage.SEVERITY_WARN,
72         null,
73         "Usuario eliminado"
74     ));
75
76     } catch (Exception e) {
77
78         FacesContext.getCurrentInstance().addMessage(null,
79             new FacesMessage(
80                 FacesMessage.SEVERITY_ERROR,
81                 null,
82                 "No se pudo eliminar"
83             ));
84     }
85 }
86
87 public void edit(User u) {
88     this.user = u;
89     editing = true;
90
91     FacesContext.getCurrentInstance().addMessage(null,
92         new FacesMessage(
93             FacesMessage.SEVERITY_INFO,
94             null,
95             "Modo edicion activado"
96         ));
97 }
98
99 public List<Role> getRoles() {
100     return roleService.findAll();
101 }
102
103 public User getUser() {
104     return user;
105 }
106
107 public void setUser(User user) {
108     this.user = user;
109 }
110
111 public boolean isEditing() {
112     return editing;
113 }
114 }

```

Listing 12: Bean UserBean como controlador JSF

Este Bean demuestra el rol del controlador dentro del patrón MVC, ya que no contiene lógica de persistencia ni acceso directo a la base de datos. Todas las operaciones son delegadas a la capa de servicios, manteniendo una clara separación de responsabilidades y permitiendo que la vista interactúe con el sistema de forma controlada.

10.5. Conversor de roles para JSF

Debido a que el formulario de usuarios utiliza un componente `selectOneMenu` que trabaja con objetos *Role*, fue necesario implementar un conversor. Este componente transforma el valor seleccionado en la vista (String) al objeto *Role* correspondiente, permitiendo que JSF asigne correctamente el rol al usuario antes de ejecutar el guardado.

```

1 package com.uteq.practicaunidadv.roleConverter;
2
3 import com.uteq.practicaunidadv.entity.Role;
4 import com.uteq.practicaunidadv.service.RoleService;
5 import org.springframework.stereotype.Component;
6 import org.springframework.web.context.WebApplicationContext;
7 import org.springframework.web.jsf.FacesContextUtils;
8
9 import javax.faces.component.UIComponent;
10 import javax.faces.context.FacesContext;
11 import javax.faces.convert.Converter;
12 import javax.faces.convert.FacesConverter;
13
14 @Component
15 @FacesConverter(value = "roleConverter", managed = true)
16 public class RoleConverter implements Converter<Role> {
17
18     private RoleService getRoleService() {
19         WebApplicationContext context =
20             FacesContextUtils.getWebApplicationContext(
21                 FacesContext.getCurrentInstance());
22         return context.getBean(RoleService.class);
23     }
24
25     @Override
26     public Role getAsObject(FacesContext context,
27                             UIComponent component,
28                             String value) {
29         if (value == null || value.isEmpty()) {
30             return null;
31         }
32         try {
33             Long id = Long.parseLong(value);
34             RoleService roleService = getRoleService();
35             return roleService.findById(id);
36         } catch (NumberFormatException e) {
37             return null;
38         }
39     }
40
41     @Override
42     public String getAsString(FacesContext context,
43                               UIComponent component,
44                               Role role) {
45         if (role == null || role.getId() == null) {
46             return "";
47         }
48         return role.getId().toString();
49     }
50 }

```

Listing 13: RoleConverter: conversion entre String e instancia Role

11. Funcionalidad CRUD desde la vista

En esta sección se describen las operaciones CRUD implementadas desde la interfaz de usuario utilizando Jakarta Server Faces (JSF). Estas funcionalidades permiten gestionar la información del sistema de forma interactiva, evidenciando la comunicación entre la vista, los Beans controladores y la capa de servicios.

11.1. Registro de usuarios

El registro de usuarios se realiza a través de un formulario web desarrollado con JSF, el cual permite ingresar los datos requeridos y enviarlos al controlador correspondiente. Al ejecutar la acción de guardado, la información es procesada por el Bean y delegada a la capa de servicios para su persistencia en la base de datos.

Desde la vista, el usuario completa los campos obligatorios y selecciona el rol correspondiente. Al presionar el botón de guardado, se invoca el método `save()` del Bean, el cual utiliza el servicio para almacenar el registro y mostrar un mensaje visual de confirmación.

ID	Nombre	Email	Acciones
1	John Silva Triviño	jsilvat2@uteq.edu.ec	Editar Eliminar

Figura 9: Formulario de registro de usuarios

El siguiente fragmento muestra un ejemplo del formulario utilizado para el registro de usuarios desde la vista JSF.

```

1 <h:form>
2
3     <h:messages showDetail="true"
4                 showSummary="false"
5                 infoClass="info"
6                 warnClass="warn"
7                 errorClass="error" />
8
9     <h:panelGrid columns="2">
10
11         <h:outputLabel value="Nombre:" />
12         <h:inputText value="#{userBean.user.name}"
13                     required="true"
14                     requiredMessage="Nombre obligatorio" />
15
16         <h:outputLabel value="Email:" />
17         <h:inputText value="#{userBean.user.email}"
18                     required="true"

```

```

19         requiredMessage="Email obligatorio"/>
20
21     <h:outputLabel value="Rol:"/>
22     <h:selectOneMenu value="#{userBean.user.role}"
23         converter="roleConverter"
24         required="true"
25         requiredMessage="Rol obligatorio">
26         <f:selectItem itemLabel="-- Seleccione un rol --"
27             itemValue="#{null}"
28             noSelectionOption="true"/>
29         <f:selectItems value="#{userBean.roles}"
30             var="rol"
31             itemLabel="#{rol.name}"
32             itemValue="#{rol}"/>
33     </h:selectOneMenu>
34
35 </h:panelGrid>
36
37 <h:commandButton
38     value="#{userBean.editing ? 'Actualizar' : 'Guardar'}"
39     styleClass="btn-save"
40     action="#{userBean.save}"/>
41
42 </h:form>

```

Listing 14: Vista users.xhtml: formulario de registro con seleccion de rol

Una vez completado el registro, el sistema muestra un mensaje de confirmación y actualiza el listado de usuarios, demostrando la correcta interacción entre la vista y las capas internas del sistema.

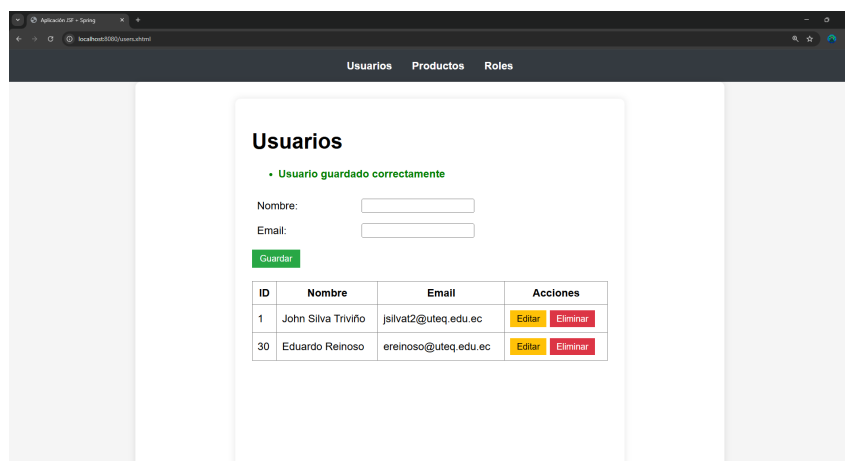


Figura 10: Mensaje de confirmación al registrar un usuario

11.2. Edición de registros

La funcionalidad de edición permite modificar la información de un usuario previamente registrado. Esta operación se activa desde el listado de usuarios, donde cada registro dispone de una opción para cargar sus datos en el formulario y habilitar el modo de edición.

Al seleccionar la opción de edición, el Bean controlador asigna el usuario seleccionado al formulario y cambia el estado interno a modo edición. Esto permite reutilizar el mismo formulario tanto para el registro como para la actualización de datos, evitando la duplicación de vistas y simplificando la interfaz.

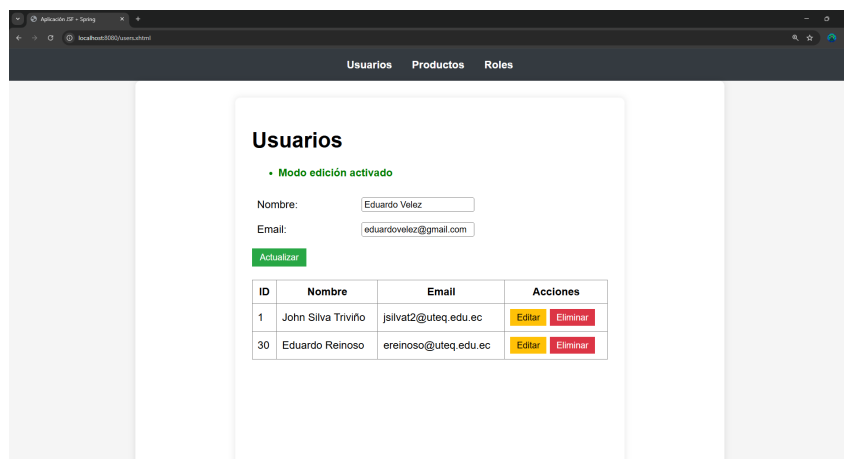


Figura 11: Opción de edición disponible en el listado de usuarios

El siguiente fragmento muestra el método utilizado en el Bean para habilitar el modo de edición y cargar los datos del usuario seleccionado.

```

1 public void edit(User u) {
2     this.user = u;
3     editing = true;
4     FacesContext.getCurrentInstance().addMessage(null,
5         new FacesMessage(
6             FacesMessage.SEVERITY_INFO,
7             null,
8             "Modo edicion activado"
9         ));
10 }

```

Listing 15: Metodo edit en UserBean

Durante el modo de edición, el botón de guardado se reutiliza para actualizar la información del usuario. Al confirmar la acción, se ejecuta nuevamente el método `save()`, el cual detecta el estado de edición y persiste los cambios en la base de datos.

Una vez completada la actualización, el sistema muestra un mensaje de confirmación y restablece el formulario a su estado inicial, demostrando una correcta gestión del flujo de edición dentro del patrón MVC.

Usuarios

• Usuario actualizado correctamente

Nombre:

Email:

ID	Nombre	Email	Acciones
1	John Silva Triviño	jsilvat2@uteq.edu.ec	Editar Eliminar
30	Eduardo Velez	eduardovelez@gmail.com	Editar Eliminar

Figura 12: Formulario de usuario en modo edición

11.3. Eliminación de registros

La eliminación de registros permite eliminar usuarios existentes del sistema de forma controlado desde la interfaz gráfica. Esta funcionalidad se encuentra disponible en el listado de usuarios, donde cada registro presenta una opción para ejecutar la acción de eliminación.

Al seleccionar la opción correspondiente, el Bean controlador recibe al usuario a eliminar y delega la operación a la capa de servicios. El servicio utiliza el repositorio para ejecutar la eliminación en la base de datos, garantizando que la vista no accederá directamente a la persistencia.

El siguiente fragmento muestra el método utilizado en el Bean para eliminar un usuario seleccionado.

```

1 public void delete(User user) {
2     try {
3         userService.delete(user);
4
5         FacesContext.getCurrentInstance().addMessage(null,
6             new FacesMessage(
7                 FacesMessage.SEVERITY_WARN,
8                 null,
9                 "Usuario eliminado"
10            ));
11
12     } catch (Exception e) {
13         FacesContext.getCurrentInstance().addMessage(null,
14             new FacesMessage(
15                 FacesMessage.SEVERITY_ERROR,
16                 null,
17                 "No se pudo eliminar"
18            ));
19     }
20 }

```

Listing 16: Metodo delete en UserBean

Una vez ejecutada la operación, el sistema muestra un mensaje visual informando el resultado de la acción y actualiza automáticamente el listado de usuarios. Esta funcionalidad demuestra la correcta comunicación entre la vista, el controlador y la capa de

servicios, manteniendo la coherencia del modelo MVC.

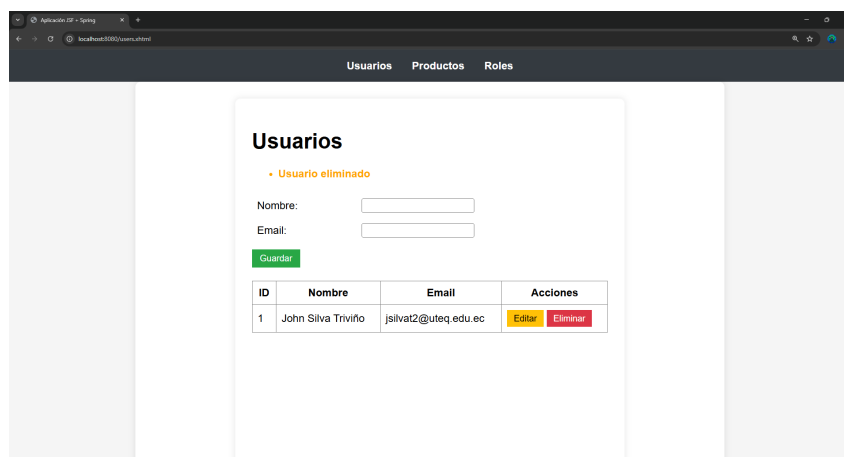


Figura 13: Mensaje de confirmación tras la eliminación de un usuario

11.4. Validación visual de campos

Con el fin de evitar el registro de información incompleta o inválida, se implementó validación visual directamente en los formularios JSF. Esta validación se realiza antes de que los datos sean procesados por el Bean controlador, mejorando la experiencia del usuario y reduciendo errores en la capa de persistencia.

Los campos obligatorios se definieron utilizando el atributo `required="true"`, el cual permite a JSF verificar que los valores hayan sido ingresados antes de ejecutar la acción de guardado. En caso de incumplimiento, el sistema muestra mensajes visuales que indican al usuario la necesidad de completar la información solicitada.

```

1 <h:inputText value="#{userBean.user.name}"
2             required="true"
3             requiredMessage="Nombre obligatorio"/>
4
5 <h:inputText value="#{userBean.user.email}"
6             required="true"
7             requiredMessage="Email obligatorio"/>
8
9 <h:selectOneMenu value="#{userBean.user.role}"
10                converter="roleConverter"
11                required="true"
12                requiredMessage="Rol obligatorio">
13     <f:selectItem itemLabel="-- Seleccione un rol --"
14                 itemValue="#{null}"
15                 noSelectionOption="true"/>
16     <f:selectItems value="#{userBean.roles}"
17                  var="rol"
18                  itemLabel="#{rol.name}"
19                  itemValue="#{rol}"/>
20 </h:selectOneMenu>

```

Listing 17: Validación de campos obligatorios en JSF

Cuando el usuario intenta enviar el formulario sin completar los campos requeridos, la vista bloquea la operación y presenta los mensajes de validación correspondientes, evitando que se ejecuten acciones innecesarias en las capas internas del sistema.

Usuarios

- Nombre obligatorio
- Email obligatorio

Nombre:

Email:

ID	Nombre	Email	Acciones
1	John Silva Triviño	jsilvat2@uteq.edu.ec	Editar Eliminar

Figura 14: Validación visual de campos obligatorios en el formulario

11.5. Gestión de roles

El módulo de roles permite registrar, editar y eliminar roles desde la interfaz JSF. Los roles registrados se utilizan posteriormente en el módulo de usuarios, debido a que la entidad *User* requiere un rol obligatorio. De esta forma, el sistema asegura consistencia en la asignación de perfiles y evita registros de usuarios sin un rol asociado.

Desde la vista `roles.xhtml` se dispone de un formulario de registro y un listado con acciones de edición y eliminación. Al igual que en el módulo de usuarios, las solicitudes se procesan en el Bean controlador, el cual delega la persistencia a la capa de servicios, manteniendo la separación de responsabilidades del patrón MVC.

Roles

Nombre del rol:

ID	Rol	Acciones
----	-----	----------

Figura 15: Módulo de roles: formulario y listado inicial

El siguiente fragmento muestra la estructura del formulario y el listado de roles en la vista.

```

1 <h:form>
2
3     <h:messages showDetail="true" showSummary="false"/>
4
5     <h:panelGrid columns="2">
6         <h:outputLabel value="Nombre del rol:"/>
7         <h:inputText value="#{roleBean.role.name}"
8                     required="true"
9                     requiredMessage="El nombre del rol es obligatorio"/
10    >
11    </h:panelGrid>
12
13    <h:commandButton
14        value="#{roleBean.editing ? 'Actualizar' : 'Guardar'}"
15        action="#{roleBean.save}"/>
16
17    <h:dataTable value="#{roleBean.roles}" var="r" border="1">
18
19        <h:column>
20            <f:facet name="header">ID</f:facet>
21            #{r.id}
22        </h:column>
23
24        <h:column>
25            <f:facet name="header">Rol</f:facet>
26            #{r.name}
27        </h:column>
28
29        <h:column>
30            <f:facet name="header">Acciones</f:facet>
31            <h:commandLink value="Editar" action="#{roleBean.edit(r)}"/>
32            \ \ | \ \
33            <h:commandLink value="Eliminar" action="#{roleBean.delete(r)
34            }"/>
35        </h:column>
36
37    </h:dataTable>
38
39 </h:form>

```

Listing 18: Vista roles.xhtml: formulario y listado de roles

A continuación, se presenta un ejemplo del Bean de roles. Este componente actúa como controlador, gestionando la interacción con la vista y delegando las operaciones a la capa de servicios.

```

1 @Component
2 @ViewScoped
3 public class RoleBean implements Serializable {
4
5     @Inject
6     private RoleService roleService;
7
8     private Role role = new Role();
9     private boolean editing = false;
10
11     public void save() {
12         try {
13             boolean wasEditing = editing;
14
15             roleService.save(role);
16
17             FacesContext.getCurrentInstance().addMessage(null,
18                 new FacesMessage(
19                     FacesMessage.SEVERITY_INFO,
20                     null,
21                     wasEditing ? "Rol actualizado correctamente"
22                         : "Rol guardado correctamente"
23                 )
24             );
25
26             role = new Role();
27             editing = false;
28
29         } catch (Exception e) {
30             FacesContext.getCurrentInstance().addMessage(null,
31                 new FacesMessage(
32                     FacesMessage.SEVERITY_ERROR,
33                     null,
34                     "Error al guardar el rol"
35                 )
36             );
37         }
38     }
39
40     public void edit(Role r) {
41         this.role = r;
42         editing = true;
43
44         FacesContext.getCurrentInstance().addMessage(null,
45             new FacesMessage(
46                 FacesMessage.SEVERITY_INFO,
47                 null,
48                 "Modo edicion activado"
49             )
50         );
51     }
52
53     public void delete(Role r) {
54         try {

```

```

55         roleService.delete(r);
56
57         FacesContext.getCurrentInstance().addMessage(null,
58             new FacesMessage(
59                 FacesMessage.SEVERITY_WARN,
60                 null,
61                 "Rol eliminado"
62             )
63         );
64
65     } catch (Exception e) {
66         FacesContext.getCurrentInstance().addMessage(null,
67             new FacesMessage(
68                 FacesMessage.SEVERITY_ERROR,
69                 null,
70                 "No se pudo eliminar el rol"
71             )
72         );
73     }
74 }
75
76 public List<Role> getRoles() {
77     return roleService.findAll();
78 }
79
80 public Role getRole() { return role; }
81 public void setRole(Role role) { this.role = role; }
82
83 public boolean isEditing() { return editing; }
84 }

```

Listing 19: Bean RoleBean: controlador JSF para roles

11.6. Gestión de productos

El módulo de productos permite realizar operaciones CRUD sobre la entidad *Product*, registrando información básica como nombre y precio. Este módulo complementa el caso de estudio, evidenciando que la estructura MVC aplicada se reutiliza en distintas entidades sin modificar el flujo general de la aplicación.

Desde la vista `products.xhtml` se presenta un formulario para registrar o actualizar productos y un listado con acciones de edición y eliminación. Al ejecutar una acción, la vista invoca métodos del Bean controlador, el cual delega la operación a la capa de servicios para su persistencia mediante el repositorio JPA.

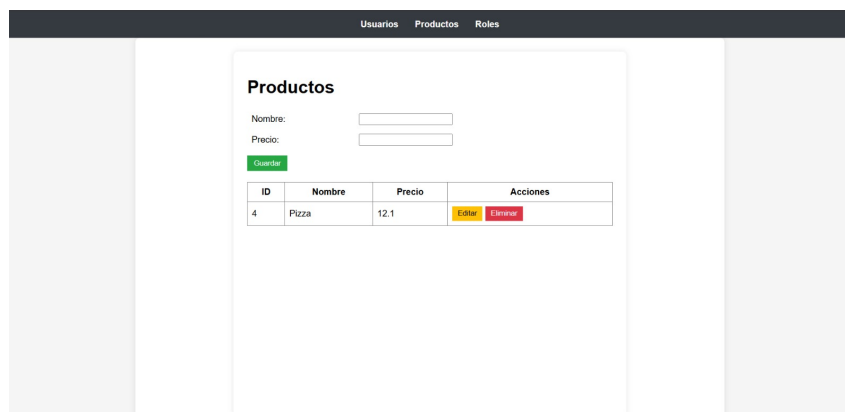


Figura 16: Módulo de productos: formulario y listado inicial

El siguiente fragmento muestra un ejemplo de formulario y listado en la vista de productos.

```

1 <h:form>
2
3     <h:messages showDetail="true" showSummary="false"/>
4
5     <h:panelGrid columns="2">
6
7         <h:outputLabel value="Nombre:"/>
8         <h:inputText value="#{productBean.product.name}"
9                     required="true"
10                    requiredMessage="El nombre es obligatorio"/>
11
12        <h:outputLabel value="Precio:"/>
13        <h:inputText value="#{productBean.product.price}"
14                    required="true"
15                    requiredMessage="El precio es obligatorio">
16            <f:validateDoubleRange minimum="0"/>
17        </h:inputText>
18
19    </h:panelGrid>
20
21    <h:commandButton
22        value="#{productBean.editing ? 'Actualizar' : 'Guardar'}"
23        action="#{productBean.save}"/>
24
25    <h:dataTable value="#{productBean.products}" var="p" border="1">
26
27        <h:column>
28            <f:facet name="header">ID</f:facet>
29            #{p.id}
30        </h:column>
31
32        <h:column>
33            <f:facet name="header">Producto</f:facet>
34            #{p.name}
35        </h:column>
36
37        <h:column>
38            <f:facet name="header">Precio</f:facet>
39            #{p.price}

```

```

40         </h:column>
41
42         <h:column>
43             <f:facet name="header">Acciones</f:facet>
44             <h:commandLink value="Editar" action="#{productBean.edit(p)}"
"/>
45             \ \ | \ \
46             <h:commandLink value="Eliminar" action="#{productBean.delete
(p)}"/>
47         </h:column>
48
49     </h:dataTable>
50
51 </h:form>

```

Listing 20: Vista products.xhtml: formulario y listado de productos

A continuación, se presenta un ejemplo del Bean correspondiente al módulo de productos, el cual mantiene la misma lógica de control aplicada en usuarios y roles.

```

1 @Component
2 @ViewScoped
3 public class ProductBean implements Serializable {
4
5     @Inject
6     private ProductService productService;
7
8     private Product product = new Product();
9     private boolean editing = false;
10
11     public void save() {
12         try {
13             boolean wasEditing = editing;
14
15             productService.save(product);
16
17             FacesContext.getCurrentInstance().addMessage(null,
18                 new FacesMessage(
19                     FacesMessage.SEVERITY_INFO,
20                     null,
21                     wasEditing ? "Producto actualizado correctamente"
22                               : "Producto guardado correctamente"
23                 )
24             );
25
26             product = new Product();
27             editing = false;
28
29         } catch (Exception e) {
30             FacesContext.getCurrentInstance().addMessage(null,
31                 new FacesMessage(
32                     FacesMessage.SEVERITY_ERROR,
33                     null,
34                     "Error al guardar el producto"
35                 )
36             );
37         }
38     }
39 }

```

```

40     public void edit(Product p) {
41         this.product = p;
42         editing = true;
43
44         FacesContext.getCurrentInstance().addMessage(null,
45             new FacesMessage(
46                 FacesMessage.SEVERITY_INFO,
47                 null,
48                 "Modo edicion activado"
49             )
50     );
51     }
52
53     public void delete(Product p) {
54         try {
55             productService.delete(p);
56
57             FacesContext.getCurrentInstance().addMessage(null,
58                 new FacesMessage(
59                     FacesMessage.SEVERITY_WARN,
60                     null,
61                     "Producto eliminado"
62                 )
63             );
64
65         } catch (Exception e) {
66             FacesContext.getCurrentInstance().addMessage(null,
67                 new FacesMessage(
68                     FacesMessage.SEVERITY_ERROR,
69                     null,
70                     "No se pudo eliminar el producto"
71                 )
72             );
73         }
74     }
75
76     public List<Product> getProducts() {
77         return productService.findAll();
78     }
79
80     public Product getProduct() { return product; }
81     public void setProduct(Product product) { this.product = product; }
82
83     public boolean isEditing() { return editing; }
84 }

```

Listing 21: Bean ProductBean: controlador JSF para productos

La validación del precio se aplica directamente en la vista, evitando registrar valores negativos. Esta verificación se ejecuta antes de que el Bean procese la solicitud, reduciendo errores y fortaleciendo el control de entrada desde la capa de presentación.

12. Reutilización y modularización

En esta sección se describe la aplicación de técnicas de reutilización y modularización en la capa de presentación, mediante el uso de Facelets y plantillas JSF. Este enfoque permite centralizar la estructura visual del sistema, reducir la duplicación de código y facilitar el mantenimiento de las vistas.

La reutilización de componentes visuales contribuye a una arquitectura más ordenada y profesional, donde los cambios en la interfaz pueden realizarse en un único lugar y reflejarse automáticamente en todas las vistas del sistema.

12.1. Arquitectura de vistas con Facelets

La arquitectura de vistas se organiza utilizando Facelets, el sistema de plantillas de JSF. Mediante esta tecnología, se definió una plantilla principal que contiene la estructura común de la aplicación, mientras que las vistas específicas solo incluyen el contenido correspondiente a cada funcionalidad.

La estructura del directorio de vistas se organizó de la siguiente manera:

```
1 webapp/  
2 |-- templates/  
3 |   |-- layout.xhtml  
4 |   |-- header.xhtml  
5 |   |-- footer.xhtml  
6 |  
7 |-- users.xhtml  
8 |-- products.xhtml  
9 |-- roles.xhtml
```

Esta organización permite separar claramente los elementos reutilizables de las vistas específicas, mejorando la claridad del proyecto.

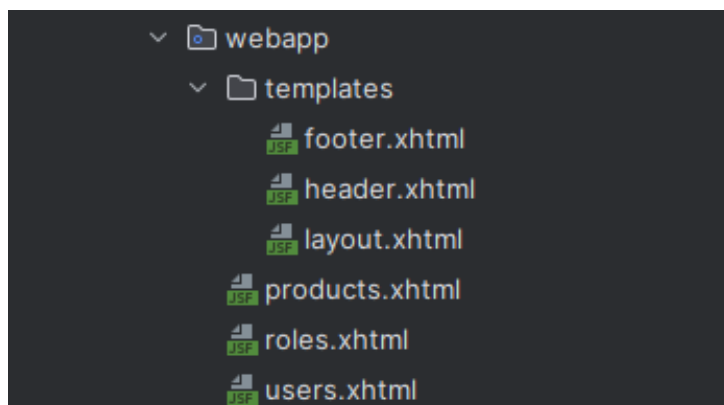


Figura 17: Estructura de vistas y plantillas Facelets

12.2. Plantilla principal layout.xhtml

La plantilla principal define la estructura base de la interfaz gráfica del sistema. En ella se incluyen los componentes reutilizables como el encabezado y el pie de página, así como una zona dinámica donde se inserta el contenido específico de cada vista.

```

1 <!DOCTYPE html>
2 <html xmlns="http://www.w3.org/1999/xhtml"
3     xmlns:h="http://xmlns.jcp.org/jsf/html"
4     xmlns:ui="http://xmlns.jcp.org/jsf/facelets">
5
6 <h:head>
7     <title>Aplicacion JSF + Spring</title>
8 </h:head>
9
10 <h:body>
11
12     <ui:include src="/templates/header.xhtml"/>
13
14     <div class="container">
15         <ui:insert name="content"/>
16     </div>
17
18     <ui:include src="/templates/footer.xhtml"/>
19
20 </h:body>
21 </html>

```

Listing 22: Plantilla principal layout.xhtml

Esta plantilla es reutilizada para todas las vistas del sistema, garantizando una apariencia uniforme y una estructura común en la aplicación.

12.3. Componentes reutilizables de interfaz

Los componentes reutilizables de interfaz se definieron en archivos independientes para el encabezado y el pie de página. Estos componentes se incluyen automáticamente en todas las vistas a través de la plantilla principal.

El encabezado contiene el menú de navegación, permitiendo el acceso a las distintas funcionalidades del sistema.

```

1 <ui:composition
2     xmlns="http://www.w3.org/1999/xhtml"
3     xmlns:h="http://xmlns.jcp.org/jsf/html"
4     xmlns:ui="http://xmlns.jcp.org/jsf/facelets">
5
6     <div class="header">
7         <h:link value="Usuarios" outcome="/users"/>
8         <h:link value="Productos" outcome="/products"/>
9         <h:link value="Roles" outcome="/roles"/>
10    </div>
11
12 </ui:composition>

```

Listing 23: Componente header.xhtml

De igual manera, el pie de página se centralizó en un único archivo, facilitando su reutilización.

```

1 <ui:composition
2   xmlns="http://www.w3.org/1999/xhtml"
3   xmlns:ui="http://xmlns.jcp.org/jsf/facelets">
4
5   <div class="footer">
6     Sistema JSF + Spring Boot - Practica Unidad V
7   </div>
8
9 </ui:composition>

```

Listing 24: Componente footer.xhtml

La aplicación de estas técnicas demuestra el uso de buenas prácticas en el desarrollo de interfaces web, favoreciendo la modularización, la reutilización de código y la mantenibilidad del sistema.

13. Pruebas y validación

En esta sección se describen las pruebas funcionales realizadas sobre el sistema desarrollado, con el objetivo de verificar el correcto funcionamiento de las operaciones CRUD y la adecuada interacción entre las capas que conforman la arquitectura MVC. Las pruebas se ejecutaron directamente desde la interfaz de usuario, validando el comportamiento del sistema en un entorno real de uso.

13.1. Pruebas de operaciones CRUD

Las pruebas funcionales se ejecutaron desde las vistas JSF con el objetivo de verificar el comportamiento de las operaciones CRUD, las validaciones de entrada y la persistencia efectiva en la base de datos. Para cada caso se registraron los datos utilizados, el resultado esperado y el resultado obtenido, incorporando evidencia mediante capturas de pantalla.

Cuadro 1: Matriz de pruebas funcionales de operaciones

ID	Módulo	Op.	Datos de entrada	Resultado esperado	Evidencia
T-01	User	C	Nombre, email único, rol válido	Registro guardado y visible en listado; mensaje de éxito	Fig. 18
T-02	User	C	Rol no seleccionado	Se bloquea el guardado y se muestra un mensaje de validación indicando que el rol es obligatorio	Fig. 19
T-03	User	U	Cambio de rol del usuario	Actualización persistente y reflejada en listado	Fig. 20
T-04	Role	C/R	Nombre de rol	Rol creado y disponible en el selector de usuarios	Fig. 21 y Fig. 22

ID	Módulo	Op.	Datos de entrada	Resultado esperado	Evidencia
T-05	Role	D	Rol asociado a usuario	El sistema impide eliminar	Fig. 23
T-06	Product	C	Nombre, precio ≥ 0	Producto guardado y listado actualizado	Fig. 24

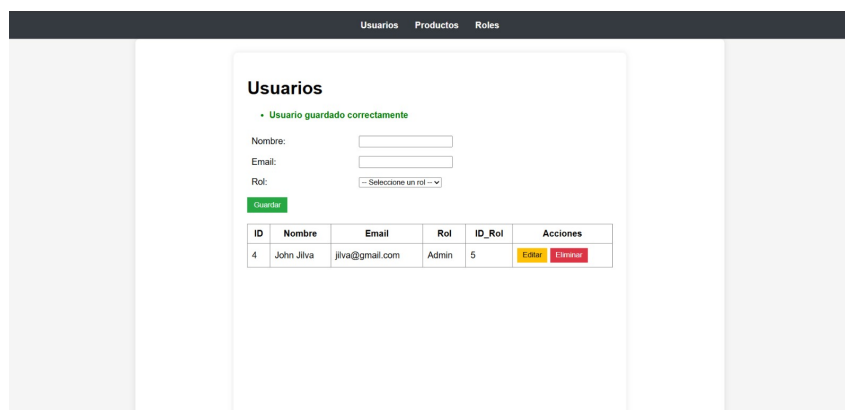


Figura 18: Prueba T-01: Registro exitoso de usuario con email único y rol válido

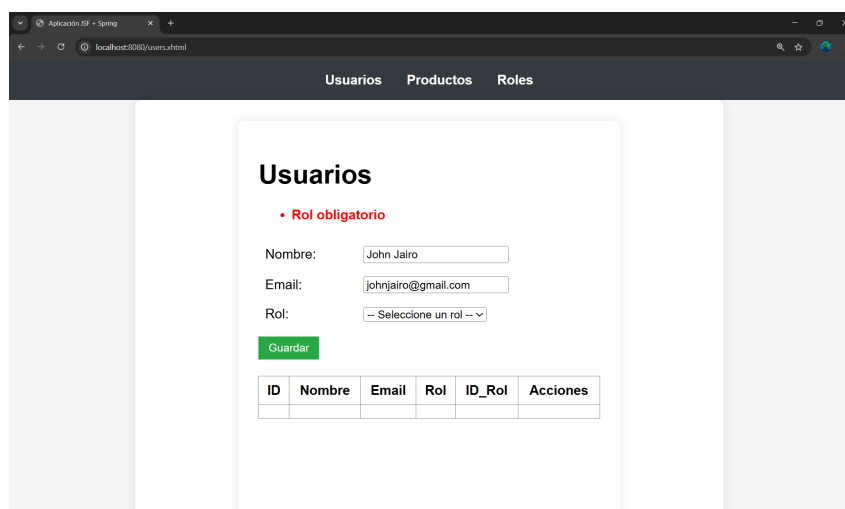


Figura 19: Prueba T-02: Validación de rol obligatorio en el registro de usuarios

Usuarios Productos Roles

Usuarios

• Usuario actualizado correctamente

Nombre:

Email:

Rol:

ID	Nombre	Email	Rol	ID_Rol	Acciones
4	John Silva	jsilva@gmail.com	Usuario	6	Editar Eliminar

Figura 20: Prueba T-03: Actualización de datos de usuario y cambio de rol

Usuarios Productos Roles

Roles

• Rol guardado correctamente

Nombre del rol:

ID	Rol	Acciones
5	Admin	Editar Eliminar
6	Usuario	Editar Eliminar
7	Cliente	Editar Eliminar

Figura 21: Prueba T-04: Creación de rol

Usuarios Productos Roles

Usuarios

Nombre:

Email:

Rol:

ID	Nombre	Rol	ID_Rol	Acciones
4	John Silva	Usuario	6	Editar Eliminar

Figura 22: Prueba T-04: Rol disponible en el módulo de usuarios

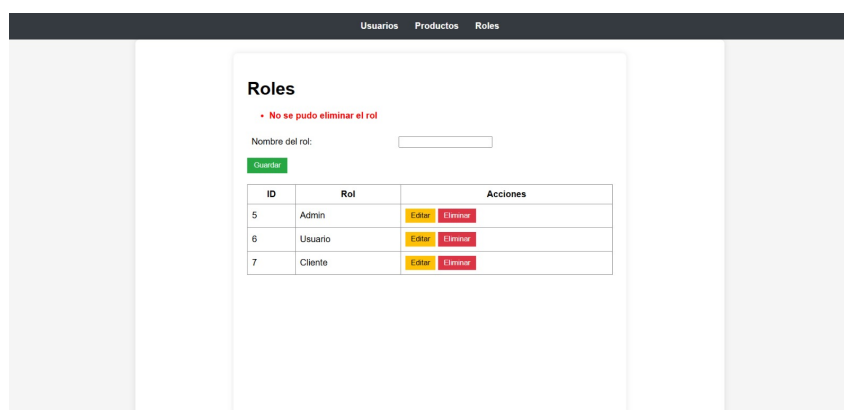


Figura 23: Prueba T-05: Restricción de eliminación de rol asociado a un usuario

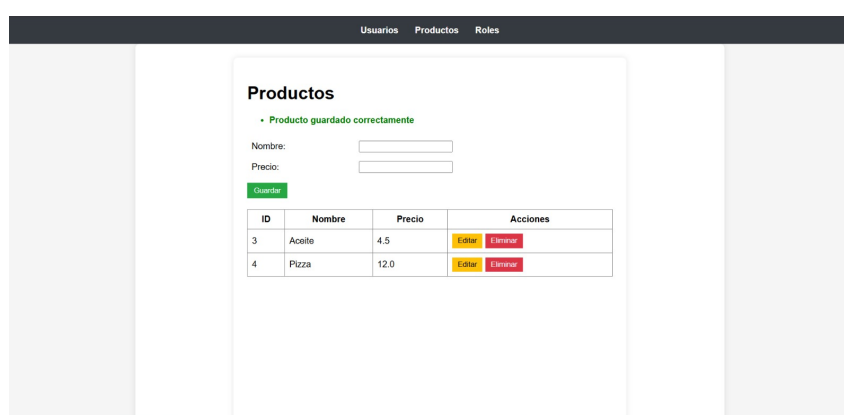


Figura 24: Prueba T-06: Registro exitoso de producto con precio válido

En todos los casos, el resultado obtenido coincidió con el resultado esperado. Las Figuras 18–24 muestran la evidencia de ejecución y los mensajes del sistema.

13.2. Verificación de interacción entre capas MVC

Se verificó que la interacción entre las capas del sistema se realice de acuerdo con el modelo MVC. En ningún caso la vista accede directamente a la base de datos, ya que todas las solicitudes son procesadas por los Beans controladores y delegadas a la capa de servicios.

La capa de servicios actúa como intermediaria, encapsulando la lógica de negocio y comunicándose con la capa de repositorios para la persistencia de datos. Esta separación de responsabilidades asegura una arquitectura clara, mantenible y alineada con buenas prácticas de desarrollo de software.

13.3. Evaluación de reutilización del código

Finalmente, se evaluó la reutilización del código implementado tanto en la capa de backend como en la capa de presentación. El uso de repositorios y servicios genéricos permitió reutilizar operaciones comunes para distintas entidades sin duplicar lógica.

De igual manera, la implementación de plantillas Facelets y componentes reutilizables facilitó la centralización de la estructura visual del sistema. Esta estrategia permitió man-

tener una interfaz consistente y simplificó la incorporación de nuevas vistas, demostrando la efectividad de la modularización aplicada en la práctica.

14. Conclusiones

El desarrollo de la práctica permitió aplicar de forma efectiva el modelo arquitectónico Modelo–Vista–Controlador (MVC) en una aplicación web funcional, evidenciando una correcta separación de responsabilidades entre la capa de presentación, la lógica de negocio y el acceso a datos. Esta organización facilitó la comprensión del flujo de la aplicación y el mantenimiento del código.

La integración de Spring Boot, Hibernate y Jakarta Server Faces demostró ser adecuada para la construcción de aplicaciones web estructuradas, permitiendo implementar operaciones CRUD de manera ordenada y consistente. El uso de repositorios y servicios genéricos contribuyó a la reutilización del código y a la reducción de duplicación en la capa de backend.

Finalmente, la aplicación de técnicas de reutilización y modularización en la interfaz, mediante el uso de plantillas Facelets y componentes compartidos, mejoró la mantenibilidad y coherencia visual del sistema. En conjunto, la práctica reforzó el uso de buenas prácticas de ingeniería de software y consolidó los conocimientos adquiridos sobre el desarrollo de aplicaciones web basadas en MVC.

Repositorio del proyecto: <https://github.com/Johnst2000/PracticaUnidadV>

Referencias

- [1] M. De Rosa, P. Ciancarini y F. Arcelli Fontana, “Empirical Assessment of the Quality of MVC Web Applications Returned by xGenerator,” *Computers*, vol. 10, n.º 2, pág. 20, 2021, Open Access, ISSN: 2073-431X. DOI: 10.3390/computers10020020. dirección: <https://www.mdpi.com/2073-431X/10/2/20>.
- [2] M. De Rosa, P. Ciancarini y F. Arcelli Fontana, “Automatic Code Generation of MVC Web Applications,” *Computers*, vol. 9, n.º 3, pág. 56, 2020, Open Access, ISSN: 2073-431X. DOI: 10.3390/computers9030056. dirección: <https://www.mdpi.com/2073-431X/9/3/56>.
- [3] D. Dobrean y L. Diosan, “Towards Predicting Architectural Design Patterns: A Machine Learning Approach,” *Computers*, vol. 11, n.º 10, pág. 151, 2022, Open Access, ISSN: 2073-431X. DOI: 10.3390/computers11100151. dirección: <https://www.mdpi.com/2073-431X/11/10/151>.
- [4] Y.-S. Choi, Y.-S. Kim y D.-H. Lee, “Transpiler-Based Architecture Design Model for Back-End Layers in Software Development,” *Applied Sciences*, vol. 13, n.º 20, pág. 11371, 2023, Open Access, ISSN: 2076-3417. DOI: 10.3390/app132011371. dirección: <https://www.mdpi.com/2076-3417/13/20/11371>.
- [5] L. F. Al-Qora’n, A. Jawarneh y J. T. Nganji, “Toward Creating Software Architects Using Mobile Project-Based Learning Model (Mobile-PBL) for Teaching Software Architecture,” *Multimodal Technologies and Interaction*, vol. 7, n.º 3, pág. 31, 2023, Open Access, ISSN: 2414-4088. DOI: 10.3390/mti7030031. dirección: <https://www.mdpi.com/2414-4088/7/3/31>.
- [6] Y. S. Lim y K. H. Lee, “Lightweight Software Architecture Evaluation for Industry: A Comprehensive Review,” *Sensors*, vol. 22, n.º 3, pág. 1252, 2022, Open Access, ISSN: 1424-8220. DOI: 10.3390/s22031252. dirección: <https://www.mdpi.com/1424-8220/22/3/1252>.
- [7] I. Ungurean y N. C. Gaitan, “A Software Architecture for the Industrial Internet of Things—A Conceptual Model,” *Sensors*, vol. 20, n.º 19, pág. 5603, 2020, Open Access, ISSN: 1424-8220. DOI: 10.3390/s20195603. dirección: <https://www.mdpi.com/1424-8220/20/19/5603>.
- [8] A. S. Abdelfattah, T. Cerny y E. Song, “A Model-Driven Architecture for Automated Deployment of Microservices,” *Applied Sciences*, vol. 11, n.º 20, pág. 9617, 2021, Open Access, ISSN: 2076-3417. DOI: 10.3390/app11209617. dirección: <https://www.mdpi.com/2076-3417/11/20/9617>.
- [9] K. Pusztai, C. Tsigkanos y S. Dustdar, “Empowering Microservices: A Deep Dive into Intelligent Application Component Placement for Optimal Response Time,” *Journal of Network and Systems Management*, vol. 32, n.º 4, pág. 85, 2024, ISSN: 1064-7570. DOI: 10.1007/s10922-024-09855-3. dirección: <https://link.springer.com/article/10.1007/s10922-024-09855-3>.
- [10] Hibernate Team, *Hibernate ORM User Guide*, Hibernate ORM 6.x Documentation, Red Hat, 2024. dirección: <https://hibernate.org/orm/documentation/>.
- [11] Jakarta EE Working Group, *Jakarta Faces 4.0 Specification*, Jakarta Faces Specification, versión 4.0, Eclipse Foundation, 2022. dirección: <https://jakarta.ee/specifications/faces/4.0/>.

-
- [12] J. Juneau y T. Telang, *Java EE to Jakarta EE 10 Recipes: A Problem-Solution Approach for Enterprise Java*. Berkeley, CA: Apress, 2022, ISBN: 978-1-4842-8079-9. DOI: 10.1007/978-1-4842-8079-9. dirección: <https://link.springer.com/book/10.1007/978-1-4842-8079-9>.
 - [13] Jakarta EE Working Group, *Jakarta EE 10 Platform Specification*, Jakarta EE 10 Platform Specification, Eclipse Foundation, 2022. dirección: <https://jakarta.ee/specifications/platform/10/>.
 - [14] R. Petcu, C. Avram y D. Dănciulescu, “Object-relational Mapping Using JPA, Hibernate and Spring Data JPA,” en *2021 International Conference on Electromechanical and Energy Systems (SIELMEN)*, IEEE, 2021, págs. 426-431. dirección: <https://ieeexplore.ieee.org/document/9481049>.
 - [15] Spring Team, *Spring Framework Reference Documentation*, Última versión: Spring Framework 6.x, Broadcom (VMware Tanzu), 2024. dirección: <https://docs.spring.io/spring-framework/reference/>.
 - [16] M. Macero y T. Telang, *Learn Microservices with Spring Boot 3: A Practical Approach Using Event-Driven Architecture, Cloud-Native Patterns, and Containerization*, 3rd. Berkeley, CA: Apress, 2023, ISBN: 978-1-4842-9757-5. DOI: 10.1007/978-1-4842-9757-5. dirección: <https://link.springer.com/book/10.1007/978-1-4842-9757-5>.
 - [17] B. Sinha, *Practical Microservices Architectural Patterns: Build Highly Scalable Distributed Applications with Spring Boot 3 and Spring Cloud*. Berkeley, CA: Apress, 2024, ISBN: 979-8-8688-1606-2. DOI: 10.1007/979-8-8688-1606-2. dirección: <https://link.springer.com/book/10.1007/979-8-8688-1606-2>.
 - [18] B. Varanasi y M. Bartkov, *Spring REST: Building Java Microservices and Cloud Applications*, 2nd. Berkeley, CA: Apress, 2022, ISBN: 978-1-4842-7477-4. DOI: 10.1007/978-1-4842-7477-4. dirección: <https://link.springer.com/book/10.1007/978-1-4842-7477-4>.
 - [19] Spring Data Team, *Spring Data JPA Reference Documentation*, Documentación oficial de Spring Data JPA, Broadcom (VMware Tanzu), 2024. dirección: <https://docs.spring.io/spring-data/jpa/reference/>.
 - [20] A. Chatterjee y A. Prinz, “Applying Spring Security Framework with KeyCloak-Based OAuth2 to Protect Microservice Architecture APIs: A Case Study,” en *Sensors*, Open Access, vol. 22, MDPI, 2022, pág. 1703. DOI: 10.3390/s22051703. dirección: <https://www.mdpi.com/1424-8220/22/5/1703>.